



UNIVERSITY OF
WEST LONDON
The **Career** University
RAK Branch Campus



DATABASES AND ANALYTICS

NorthStar: Integrated Analytics and Data



GitHub Repository: https://github.com/Amaan449/DBA_ASSIGNMENT

Student Id: 32147390

Student Name: Mohammed Amaan Danish

Email id: 32147390@student.uwl.ac.uk

Module: Databases and Analytics

Module Tutor: Mr. Rajan

Contents

NorthStar: Integrated Analytics and Data	1
List Of Figures	4
Executive Summary	6
1. Introduction	7
1.1 Business Context	7
1.2 Business Objectives.....	8
2. Data Understanding and Dataset Overview	9
2.1 Source Datasets.....	9
2.2 Data Quality Challenges	13
3. Data Cleaning, Preprocessing, and Integration	13
3.1 Column Standardisation.....	13
3.2 Zone Label Standardisation	13
3.3 Datetime and Numeric Type Enforcement.....	15
3.4 Merge: Building the Integrated Master Dataset	16
4. SQL within R Analysis	19
4.1 Analytical Setup.....	19
4.2 Query 1: Delay Performance by Pickup Zone.....	19
4.3 Query 2: Complaint Intensity by Service Type	20
4.4 Query 3: High-Override Routes	21
4.5 Query 4: Hub-Level Performance.....	22
4.6 Query 5: Driver Risk Profile	23
4.7 Query 6: Repeat-Complaint Customers	24
4.8 Query 7: Vehicle Maintenance and Delivery Performance	25
4.9 Query 8: Monthly Operational Trend.....	26
4.10 Query 9: INSERT, UPDATE, and DELETE Demonstrations	27
4.11 Mathematical and Aggregate Functions	28
4.12 Statistical Validation	29
5. R Analytics and Visualisation.....	32
5.1 Delay Distribution by Zone	32
5.2 Failure Rate by Service Type.....	32
5.3 Hub Performance Heatmap	33
5.4 Monthly Delay Rate Trend.....	34
5.5 Correlation Matrix.....	34
5.6 Scatterplot: Order Value vs. Delay by Service Type.....	35
5.7 Violin Plot: Route Override by Employment Type.....	36

5.8 Bar Chart: Proof-of-Completion Missing Rate by Zone	37
6. Python Data Processing and Operational Analytics	38
6.1 Feature Engineering	38
6.2 Feature Engineering Limitations and Clarifications.....	40
6.3 NumPy and Pandas Operational Analysis	41
6.4 Hub Efficiency Score	41
6.5 Customer Risk Segment Analysis	42
6.6 Top 10 Highest-Risk Routes	43
6.7 Fleet Health Analysis.....	44
7. Pairwise Numerical Visualisation	44
7.1 Pairplot – Core Operational Metrics	44
7.2 Correlation Heatmap.....	45
7.3 Delay Category Stacked Bar by Zone.....	46
7.4 Route Risk vs. Hub Failure Rate Scatter.....	47
8. MongoDB Atlas Design and Implementation.....	49
8.1 NoSQL Rationale	49
8.2 Document Structure: customer_cases Collection.....	49
8.3 CRUD Operations	50
8.4 Aggregation Pipeline	52
9. NoSQL Schema Design	53
9.1 Design Principle: Embed or Reference?	53
9.2 Collections Architecture.....	53
9.3 What Remains Relational	53
10. Query Optimisation and Performance Tuning	54
10.1 Baseline Performance: COLLSCAN	54
10.2 Index Strategy	54
10.3 Performance Improvement.....	55
10.4 Index Design Justification.....	56
11. MongoDB Analytical Insights	56
11.1 Insight 1: Hub Escalation Distribution.....	56
11.2 Insight 2: API Latency by Zone	58
11.3 Insight 3: Proof of Completion × Complaint Interaction	59
12. Discussion and Recommendations	61
12.1 Integrated Findings	61
12.2 Recommendations	62
13. Conclusion.....	63

14. Reference	64
---------------------	----

List Of Figures

Figure 1 Google Colab – repository clone and directory structure output.....	9
Figure 2 Dataset load output showing shapes for all nine source files.....	10
Figure 3 Showing unique zone values before standardisation.....	11
Figure 4 Unique zone values after standardisation.....	12
Figure 5 Missing value counts per table before and after cleaning	14
Figure 6 Data type conversion finished.....	15
Figure 7 Merge – intermediate output after adding deliveries and customers (shape and first rows)	17
Figure 8 Merge – incident fields added.....	17
Figure 9 Merge – aggregated complaints and app events	17
Figure 10 Merge – final verification: shape (1,250 rows, 59 columns), duplicate check, and row	18
Figure 11 SQL Query 1 average delay and failure rate by pickup zone	19
Figure 12 SQL Query 2 complaint and delay rates by service type	20
Figure 13 SQL Query 3 top 10 override routes.....	21
Figure 14 SQL Query 4 hub level performance table	22
Figure 15 SQL Query 5 top 15 high override drivers.....	23
Figure 16 SQL Query 6 repeat complaint customer list	24
Figure 17 SQL Query 7 vehicle maintenance vs. delay rates.....	25
Figure 18 SQL Query 8 monthly trend table	26
Figure 19 SQL Query 9 INSERT, UPDATE, DELETE demonstrations.....	27
Figure 20 SQL aggregate operational summary statistics and cost per km analysis	28
Figure 21 R t test comparing Central vs. South zone delays	29
Figure 22 R chi square test for service type vs. delay status.....	30
Figure 23 Bar Chart Delay rate by service type	31
Figure 24 R boxplot delay distribution by pickup zone	32
Figure 25 R bar chart failure rate by service type	33
Figure 26 R heatmap hub performance metrics	33
Figure 27 R line chart monthly delay rate trend with regression overlay	34
Figure 28 R correlation matrix ten operational variables.....	35
Figure 29 R scatterplot order value vs. delay by service type	36
Figure 30 R violin plot route override frequency by employment type.....	37
Figure 31 R bar chart proof of completion missing rate by zone	37
Figure 32 Python output feature engineering summary statistics table	40
Figure 33 Python output NumPy descriptive statistics array	41
Figure 34 Python hub efficiency score ranking	42
Figure 35 Python customer risk segment analysis table	43
Figure 36 Python top 10 highest risk routes by composite score	43
Figure 37 Python fleet health summary by vehicle type	44
Figure 38 Python pair plot six operational variables coloured by service type	45
Figure 39 Python correlation heatmap fourteen operational and engineered variables	46
Figure 40 Python stacked bar chart delay category distribution by pickup zone.....	47
Figure 41 Python scatter plot route risk score vs. hub failure rate, coloured by delay hours.....	48
Figure 42 MongoDB Atlas connection confirmation.....	49
Figure 43 MongoDB Atlas customer_cases collection document structure	50

Figure 44 Inserted 1250 document into customer_cases.....	50
Figure 45 Python FIND query results for all three queries.....	51
Figure 46 MongoDB update_many matched and modified counts.....	52
Figure 47 Deleted document with null delivery_id.....	52
Figure 48 MongoDB aggregation pipeline result by zone	52
Figure 49 MongoDB explain() before indexing showing COLLSCAN stage.....	54
Figure 50 MongoDB all five indexes created on customer_cases collection	55
Figure 51 MongoDB explain() after indexing showing IXSCAN stage	56
Figure 52 MongoDB hub escalation aggregation pipeline result	57
Figure 53 MongoDB API latency by zone pipeline result	58
Figure 54 MongoDB proof × complaint aggregation (table)	59
Figure 55 MongoDB proof × complaint (bar chart).....	60

Executive Summary

This research examines the issues facing NorthStar Urban Mobility and Logistics, a regional operator whose service performance has been declining despite the company's continued growth in demand. We integrated three analytical tools—SQL (run through R), Python for data processing, and MongoDB Atlas for document storage—to investigate the delays, complaints, and inefficiencies across their seven zones and five service kinds.

The central zone is bearing the brunt of the failures, which is the main finding. With a failure rate of 34.5% and an average delivery delay of three hours, it is more than three times worse than the South zone. Medical and business services are also failing at significantly greater percentages (27.3% and 32.1%, respectively). The official complaints system really detects the possibility of silent churn among NorthStar's most important customers. In the meanwhile, Central Zone routes are more likely to have manual route overrides, which are associated with increased fuel expenses and lengthier delays. This implies that drivers are not simply ignoring the route planning algorithm, but rather that it is consistently failing.

Additionally, NorthStar's operational event data—such as app interactions, complaint histories, route exceptions, and driver issue logs—cannot be adequately handled by its present relational database. These typically contain complex structures, changing schemas, and frequent updates. However, you can achieve effective case-level searching with a MongoDB Atlas solution that uses embedded documents, and at production scale, compound indexing reduces the potential execution time for operational queries by two orders of magnitude.

Thus, the report concludes with seven useful suggestions. Route infrastructure, client retention, proof of capture enforcement, proactive EV fleet maintenance, contract driver onboarding, implementing a hybrid database architecture, and expanding predictive analytics are all included.

1. Introduction

1.1 Business Context

NorthStar Urban Mobility and Logistics provides services in a number of major UK cities, including warehouse dispatch, medical couriers, business freight, public shuttles, last-mile parcel delivery, retail logistics, and electric vehicle charging stations. Over the course of the last six years, the company has expanded rapidly, primarily through mergers and acquisitions, each of which brought with it unique systems, naming conventions, and data cultures.

As a result, you have a really disjointed technological arrangement. Tech teams continue to argue that traditional relational databases simply cannot handle the volume and variety of operational data coming through, customer experience teams witness an increase in complaints, operations teams examine throughput numbers that appear fine on paper, and finance teams suspect certain routes are quietly losing money.

The people in charge have found some things that are worrying. They do not make sense when we look at each part of the company on its own. Let us look at complaints from customers for example. Some customers keep saying that the services they got did not work or that people came late. Then we have one system that says these things were done and another system that says they were handled in a special way.

When drivers choose other routes than the route they were given this is also a problem. In some areas drivers are doing this a lot. We do not know if this is because the route was not planned well or if the roads were really bad or if the drivers are just trying to get around the targets they have to meet.

There is also a problem with how we use our assets. The system that watches our assets says they are not being used much. The records from the places where we charge them say they are being used a lot. This means that there might be a problem, with how we're counting or understanding what is going on with the assets. The senior management team is looking at customer complaints and manual route overrides and asset usage data to figure out what is going on with the company.

1.2 Business Objectives

The investigation addresses five core business questions:

Objective 1 – Geographic and service concentration of failure. Which zones, hubs, and service types generate disproportionate delays, failures, and complaints? Are these concentrated or distributed across the network?

Objective 2 – Driver and route behaviour as failure drivers. Is manual route override behaviour a symptom of poor planning, poor training, or poor tooling? Does employment type or experience level explain variance in service quality?

Objective 3 – Vehicle fleet reliability and operational impact. Do vehicles in poor condition or under maintenance produce worse delivery outcomes? Is the EV fleet's growing share creating new maintenance risks?

Objective 4 – Complaint data accuracy as service failure proxy. Are formal complaints a reliable indicator of operational quality, or is there hidden dissatisfaction among customer segments that do not escalate?

Objective 5 – Hybrid architecture suitability assessment. Which data belongs in structured relational databases, and which is better represented as nested documents in MongoDB Atlas for operational querying at scale?

2. Data Understanding and Dataset Overview

2.1 Source Datasets

Nine source datasets were provided, totalling 4,477 records across all tables before merging. After cleaning, standardisation, and left-join merging into a single integrated dataframe, the master dataset contains **1,250 rows and 59 columns**, representing one row per order with all associated delivery, customer, driver, vehicle, hub, incident, complaint, and app event attributes attached.

```
!git clone https://github.com/Amaan449/DBA_ASSIGNMENT.git
%cd DBA_ASSIGNMENT

Cloning into 'DBA_ASSIGNMENT'...
remote: Enumerating objects: 34, done.
remote: Counting objects: 100% (34/34), done.
remote: Compressing objects: 100% (24/24), done.
remote: Total 34 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (34/34), 108.40 KiB | 1.67 MiB/s, done.
Resolving deltas: 100% (2/2), done.
/content/DBA_ASSIGNMENT/DBA_ASSIGNMENT
```

Figure 1 Google Colab – repository clone and directory structure output

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime
import warnings
warnings.filterwarnings('ignore')

# Load all CSV files
customers = pd.read_csv('Datasets/customers.csv')
orders = pd.read_csv('Datasets/orders.csv')
deliveries = pd.read_csv('Datasets/deliveries.csv')
drivers = pd.read_csv('Datasets/drivers.csv')
vehicles = pd.read_csv('Datasets/vehicles.csv')
hubs = pd.read_csv('Datasets/hubs.csv')
incidents = pd.read_csv('Datasets/incidents.csv')
complaints = pd.read_csv('Datasets/complaints.csv')
app_events = pd.read_csv('Datasets/app_events.csv')

# Store in dictionary for easy access
datasets = {
    "customers": customers,
    "orders": orders,
    "deliveries": deliveries,
    "drivers": drivers,
    "vehicles": vehicles,
    "hubs": hubs,
    "incidents": incidents,
    "complaints": complaints,
    "app_events": app_events
}

# Display shape of each dataset
for name, df in datasets.items():
    print(f"{name}: {df.shape}")

```

```

customers: (650, 9)
orders: (1250, 11)
deliveries: (950, 13)
drivers: (170, 8)
vehicles: (120, 8)
hubs: (8, 5)
incidents: (280, 7)
complaints: (320, 10)
app_events: (640, 10)

```

Figure 2 Dataset load output showing shapes for all nine source files

The dataset spans multiple UK operational zones – Central, North, South, East, West, Riverside, and Airport – and covers five service types: Passenger, Parcel, Retail, Business, and Medical. The vehicle fleet includes EVs, Hybrids, Cargo Vans, and Diesel vehicles.

```
# Standardize all column names to lowercase and remove extra whitespace
for name in datasets:
    datasets[name].columns = datasets[name].columns.str.strip().str.lower()# Standardize all column names to lowercase and remove extra whitespace
for name in datasets:
    datasets[name].columns = datasets[name].columns.str.strip().str.lower()

# For each specified zone column, print its unique non-missing values across all datasets that contain it
for col in ["home_zone", "pickup_zone", "dropoff_zone", "base_zone", "zone", "zone_context"]:
    print(f"\n--- {col} ---")
    for name, df in datasets.items():
        if col in df.columns:
            print(name, df[col].dropna().unique())

...

--- home_zone ---
customers ['North' 'AIRPORT' 'East' 'CENTRAL' 'Riverside' 'WEST' 'north' 'South'
'West' 'Ctr' 'Airport' 'EAST' 'SOUTH' 'RiverSide' 'NORTH' 'Central']

--- pickup_zone ---
orders ['Airport' 'North' 'West' 'RiverSide' 'Riverside' 'CENTRAL' 'NORTH'
'South' 'WEST' 'East' 'Ctr' 'SOUTH' 'Central' 'AIRPORT' 'north' 'EAST']

--- dropoff_zone ---
orders ['South' 'AIRPORT' 'North' 'SOUTH' 'East' 'Airport' 'Riverside' 'north'
'Central' 'Ctr' 'CENTRAL' 'NORTH' 'RiverSide' 'West' 'EAST' 'WEST']

--- base_zone ---
drivers ['AIRPORT' 'Central' 'Airport' 'north' 'CENTRAL' 'North' 'SOUTH' 'West'
'East' 'NORTH' 'EAST' 'WEST' 'Riverside' 'South' 'RiverSide' 'Ctr']

--- zone ---
hubs ['North' 'South' 'East' 'West' 'Central' 'Airport' 'Riverside']

--- zone_context ---
app_events ['north' 'SOUTH' 'Airport' 'CENTRAL' 'West' 'South' 'Ctr' 'North' 'WEST'
'AIRPORT' 'RiverSide' 'NORTH' 'EAST' 'Riverside' 'East' 'Central']
```

Figure 3 Showing unique zone values before standardisation

```

# Define a mapping from common zone spellings to standardized zone names
zone_map = {
    "north": "North",
    "south": "South",
    "east": "East",
    "west": "West",
    "central": "Central",
    "ctr": "Central",
    "riverside": "Riverside",
    "airpor": "Airport",
    "airport": "Airport"
}

# Function to clean and standardize zone values using the zone_map
def clean_zone(x):
    if pd.isna(x):
        return np.nan
    x = str(x).strip()
    key = x.lower()
    return zone_map.get(key, x.title())

# Apply the clean_zone function to standardize zone values in relevant columns
if "home_zone" in customers.columns:
    customers["home_zone"] = customers["home_zone"].apply(clean_zone)

if "pickup_zone" in orders.columns:
    orders["pickup_zone"] = orders["pickup_zone"].apply(clean_zone)

if "dropoff_zone" in orders.columns:
    orders["dropoff_zone"] = orders["dropoff_zone"].apply(clean_zone)

if "base_zone" in drivers.columns:
    drivers["base_zone"] = drivers["base_zone"].apply(clean_zone)

if "zone" in hubs.columns:
    hubs["zone"] = hubs["zone"].apply(clean_zone)

if "zone_context" in app_events.columns:
    app_events["zone_context"] = app_events["zone_context"].apply(clean_zone)

# Show unique non-missing values for each zone column after cleaning
for col in ["home_zone", "pickup_zone", "dropoff_zone", "base_zone", "zone", "zone_context"]:
    print(f"\n--- {col} AFTER CLEANING ---")
    for name, df in {
        "customers": customers,
        "orders": orders,
        "drivers": drivers,
        "hubs": hubs,
        "app_events": app_events
    }.items():
        if col in df.columns:
            print(name, df[col].dropna().unique())

```

```

...

--- home_zone AFTER CLEANING ---
customers ['North' 'Airport' 'East' 'Central' 'Riverside' 'West' 'South']

--- pickup_zone AFTER CLEANING ---
orders ['Airport' 'North' 'West' 'Riverside' 'Central' 'South' 'East']

--- dropoff_zone AFTER CLEANING ---
orders ['South' 'Airport' 'North' 'East' 'Riverside' 'Central' 'West']

--- base_zone AFTER CLEANING ---
drivers ['Airport' 'Central' 'North' 'South' 'West' 'East' 'Riverside']

--- zone AFTER CLEANING ---
hubs ['North' 'South' 'East' 'West' 'Central' 'Airport' 'Riverside']

--- zone_context AFTER CLEANING ---
app_events ['North' 'South' 'Airport' 'Central' 'West' 'Riverside' 'East']

```

Figure 4 Unique zone values after standardisation

2.2 Data Quality Challenges

Before cleaning, this dataset had a few quirks that made it tricky to analyse properly. Zone labels, for example, appeared in at least five different formats across tables – think “north”, “NORTH”, “North”, “ctr”, “central” – all meaning the same thing but not matching up. Several columns stored numeric values as strings, and date formats were all over the place. Complaint and app event data also came in at a different level of detail than the orders, which meant we had to aggregate them before merging. The upshot is that a naive merge would've given us misleading results – so the data engineering stage wasn't optional; it was a prerequisite for any reliable conclusions.

3. Data Cleaning, Preprocessing, and Integration

3.1 Column Standardisation

We standardised every column name across all dataframes by converting them to lowercase and stripping any leading or trailing whitespace. That might sound minor, but different source systems had exported columns with inconsistent capitalisation – and that's a classic, silent cause of join failures if you're not careful.

3.2 Zone Label Standardisation

The biggest headache was the zone labels. Fields like `pickup_zone`, `dropoff_zone`, `home_zone`, `base_zone`, `zone`, and `zone_context` all referred to the same seven geographic areas, but they came in loads of inconsistent formats. So we wrote a custom mapping function to squash everything down to one canonical format. That eliminated a whole category of grouping error – the kind that would have made any zone-level summary completely misleading.

.

```

for name, df in datasets.items():
    print(f"\n--- {name.upper()} MISSING VALUES ---")
    print(df.isna().sum()[df.isna().sum() > 0])

```

```

--- CUSTOMERS MISSING VALUES ---
loyalty_score      20
preferred_channel   13
dtype: int64

--- ORDERS MISSING VALUES ---
booking_channel     25
dtype: int64

--- DELIVERIES MISSING VALUES ---
delivery_completed_at      19
customer_rating_post_delivery  14
dtype: int64

--- DRIVERS MISSING VALUES ---
training_score    7
dtype: int64

--- VEHICLES MISSING VALUES ---
battery_health_pct     4
maintenance_status     120
dtype: int64

--- HUBS MISSING VALUES ---
Series([], dtype: int64)

--- INCIDENTS MISSING VALUES ---
resolved_hours    17
dtype: int64

--- COMPLAINTS MISSING VALUES ---
compensation_amount    16
dtype: int64

--- APP_EVENTS MISSING VALUES ---
order_id    144
dtype: int64

```

Figure 5 Missing value counts per table before and after cleaning

3.3 Datetime and Numeric Type Enforcement

Six datetime columns across five tables were parsed using `pd.to_datetime` with `errors='coerce'`, converting unparseable values to `NaT`. Seventeen numeric columns across seven tables were coerced using `pd.to_numeric` with `errors='coerce'`, including operational metrics such as `route_distance_km`, `manual_route_override_count`, `fuel_or_charge_cost`, `driver_rating`, `training_score`, and `battery_health_pct`.

```
# Convert datetime columns
date_cols = {
    "customers": ["signup_date"],
    "orders": ["order_created_at"],
    "deliveries": ["dispatch_time", "delivery_completed_at"],
    "complaints": ["created_at"],
    "incidents": ["reported_at"],
    "app_events": ["event_timestamp"]
}

dfs = {
    "customers": customers,
    "orders": orders,
    "deliveries": deliveries,
    "complaints": complaints,
    "incidents": incidents,
    "app_events": app_events
}

for name, cols in date_cols.items():
    df = dfs[name]
    for col in cols:
        if col in df.columns:
            df[col] = pd.to_datetime(df[col], errors="coerce")

# Convert numeric columns
numeric_conversions = {
    "customers": ["loyalty_score", "app_engagement_score"],
    "orders": ["promised_window_hours", "order_value", "special_handling_flag"],
    "deliveries": ["route_distance_km", "manual_route_override_count",
                  "proof_of_completion_missing", "customer_rating_post_delivery",
                  "fuel_or_charge_cost"],
    "drivers": ["years_experience", "training_score", "driver_rating", "active_flag"],
    "vehicles": ["battery_health_pct", "maintenance_status"],
    "complaints": ["compensation_amount"],
    "incidents": ["resolved_hours"],
    "app_events": ["api_latency_ms", "success_flag"]
}

dfs_num = {
    "customers": customers,
    "orders": orders,
    "deliveries": deliveries,
    "drivers": drivers,
    "vehicles": vehicles,
    "complaints": complaints,
    "incidents": incidents,
    "app_events": app_events
}

for name, cols in numeric_conversions.items():
    df = dfs_num[name]
    for col in cols:
        if col in df.columns:
            df[col] = pd.to_numeric(df[col], errors="coerce")

print("Data type conversion finished.")
```

Data type conversion finished.

Figure 6 Data type conversion finished

3.4 Merge: Building the Integrated Master Dataset

I combined the nine source datasets into a single master table, *orders_full*, using a hub-and-spoke left join approach with the orders table acting as the central spine. This method works well because it preserves all 1,250 orders, even where related data is missing—for example, when there’s no delivery record, no assigned driver, or no logged complaint. The overall process is outlined below.

Step 1: Attach delivery, customer, hub, driver, and vehicle data

I then applied a series of sequential left joins to bring in key attributes, including dispatch and completion times, status indicators, manual overrides, fuel cost, and a proof-missing flag. This was followed by the addition of customer demographics, hub information, driver attributes, and vehicle characteristics.

```
# Merging attach delivery, customer, hub, driver, and vehicle data

# Enrich orders with delivery information by merging on order_id
orders_full = orders.merge(
    deliveries,
    on="order_id",
    how="left",
    suffixes=("_order", "_del")
)

# Enrich the orders-deliveries table with selected customer features by merging on customer_id
orders_full = orders_full.merge(
    customers[[
        "customer_id",
        "age",
        "home_zone",
        "customer_type",
        "loyalty_score",
        "app_engagement_score",
        "preferred_channel",
        "account_status"
    ]],
    on="customer_id",
    how="left"
)

# Add hub details (hub_name, zone, hub_type, capacity_score) to orders_full by merging on hub_id
orders_full = orders_full.merge(
    hubs[["hub_id", "hub_name", "zone", "hub_type", "capacity_score"]],
    on="hub_id",
    how="left"
)

# Enrich orders with driver attributes by merging on driver_id
orders_full = orders_full.merge(
    drivers[[
        "driver_id",
        "base_zone",
        "employment_type",
        "years_experience",
        "training_score",
        "driver_rating",
        "shift_preference",
        "active_flag"
    ]],
    on="driver_id",
    how="left"
)

# Add vehicle details (e.g., vehicle_type, battery_health_pct, maintenance_status) to orders by merging on vehicle_id
orders_full = orders_full.merge(
    vehicles,
    on="vehicle_id",
    how="left"
)

# OUTPUT ONLY AFTER ALL JOINS ARE COMPLETED
print(orders_full.shape)
orders_full.head()
```


(1250, 48)

	order_id	customer_id	service_type	order_created_at	promised_window_hours	pickup_zone	dropoff_zone	priority_level	order_value	booking_channel	...	driver_rating	shift_preference	active_flag	vehicle_type	assigned_name	commission_date	battery_health_pct	odometer_km	maintenance_status	telematics_version
0	C00001	C0002	Passenger	2024-08-20 14:43:00	6	Airport	South	Medium	126.65	App	...	4.70	Morning	1.0	Hybrid	WEST	2024-08-19 01:10:00	93.8	98472.0	NaN	v2.0
1	C00002	C0450	Passenger	2024-05-14 22:16:00	24	North	Airport	Low	109.30	App	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	C00003	C0161	Passenger	2025-09-02 14:37:00	4	West	Airport	High	33.50	Phone	...	5.97	Morning	1.0	CargoVan	East	2024-08-23 11:24:00	60.1	31878.0	NaN	v2.0
3	C00004	C0020	Parcel	2025-01-11 17:15:00	2	Riverside	North	Medium	10.04	App	...	4.81	Evening	1.0	Diesel	AIRPORT	2025-05-17 06:52:00	68.6	78488.0	NaN	v2.2
4	C00005	C0558	Retail	2025-02-17 19:32:00	12	Riverside	South	Low	125.58	Phone	...	4.52	Morning	1.0	Hybrid	AIRPORT	2024-05-28 06:33:00	NaN	191353.0	NaN	v2.1

5 rows * 48 columns

Figure 7 Merge – intermediate output after adding deliveries and customers (shape and first rows)

Step 2: Attach incident data:-

Incident type, severity, resolved hours were left-joined on delivery_id.

```

# Attach Incident Information (incident_type, severity, resolved_hours) to orders by merging on delivery_id
incidents_subset = incidents[
    "delivery_id",
    "incident_type",
    "severity",
    "resolved_hours"
]
incidents_subset = incidents_subset[~incidents_subset["delivery_id"].isnull()]

orders_full = orders_full.merge(
    incidents_subset,
    on="delivery_id",
    how="left"
)

print(orders_full.shape)
orders_full.head()

```

(1250, 51)

order_id	customer_id	service_type	order_created_at	promised_window_hours	pickup_zone	dropoff_zone	priority_level	order_value	booking_channel	...	vehicle_type	assigned_name	commission_date	battery_health_pct	odometer_km	maintenance_status	telematics_version	incident_type	severity	resolved_hours	
0	C00001	C0002	Passenger	2024-08-20 14:43:00	6	Airport	South	Medium	126.65	App	...	Hybrid	WEST	2024-08-19 01:10:00	93.8	98472.0	NaN	v2.0	NaN	NaN	NaN
1	C00002	C0450	Passenger	2024-05-14 22:16:00	24	North	Airport	Low	109.30	App	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	C00003	C0161	Passenger	2025-09-02 14:37:00	4	West	Airport	High	33.50	Phone	...	CargoVan	East	2024-08-23 11:24:00	60.1	31878.0	NaN	v2.0	NaN	NaN	NaN
3	C00004	C0020	Parcel	2025-01-11 17:15:00	2	Riverside	North	Medium	10.04	App	...	Diesel	AIRPORT	2025-05-17 06:52:00	68.6	78488.0	NaN	v2.2	NaN	NaN	NaN
4	C00005	C0558	Retail	2025-02-17 19:32:00	12	Riverside	South	Low	125.58	Phone	...	Hybrid	AIRPORT	2024-05-28 06:33:00	NaN	191353.0	NaN	v2.1	NaN	NaN	NaN

5 rows * 51 columns

Figure 8 Merge – incident fields added

Step 3: Aggregate and attach complaints and app events

Complaints grouped by order_id produced complaint_count, any_high_severity, avg_compensation. App events grouped by order_id produced app_event_count, unique_event_types, avg_api_latency_ms. Both were left-joined.

```
# Aggregate complaint data by order_id
complaints_by_order = (
    complaints
    .groupby("order_id")
    .agg(
        complaint_count=("complaint_id", "count"),
        most_common_severity=("severity", lambda x: x.mode().iloc[0] if len(x.mode()) > 0 else None)
    )
    .reset_index()
)

# Attach aggregated complaint features to orders by merging with complaints_by_order on order_id
orders_full = orders_full.merge(
    complaints_by_order,
    on="order_id",
    how="left"
)

# Aggregate app event data by order_id to create summary features for each order
app_events_by_order = (
    app_events.dropna(subset=["order_id"])
    .groupby("order_id")
    .agg(
        app_event_count=("event_id", "count"),
        last_app_event=("event_timestamp", "max"),
        unique_event_types=("event_type", "nunique"),
        avg_api_latency=("api_latency_ms", "mean")
    )
    .reset_index()
)

app_events_by_order["avg_api_latency"] = app_events_by_order["avg_api_latency"].round(2)

# Attach aggregated app event features to orders_full
orders_full = orders_full.merge(
    app_events_by_order,
    on="order_id",
    how="left"
)

# OUTPUT ONLY AFTER ALL JOINS ARE COMPLETED
print(orders_full.shape)
orders_full.head()
```

(1250, 54)

order_id	customer_id	service_type	order_created_at	promised_window_hours	pickup_zone	dropoff_zone	priority_level	order_value	booking_channel	...	battery_health_pct	odometer_km	maintenance_status	telematics_version	complaint_count	most_common_severity	app_event_count	last_app_event	unique_event_types	avg_api_latency		
0	C00001	C0002	Passenger	2024-08-20 14:43:00	6	Airport	South	Medium	126.65	App	...	93.8	98472.0	NaN	v2.0	NaN	NaN	NaN	1.0	2024-08-02 12:35:00	1.0	204.0
1	C00002	C0459	Passenger	2024-05-14 22:16:00	24	North	Airport	Low	109.30	App	...	NaN	NaN	NaN	NaN	NaN	NaN	1.0	2025-08-28 03:54:00	1.0	244.0	
2	C00003	C0161	Passenger	2025-09-02 14:37:00	4	West	Airport	High	33.50	Phone	...	60.1	31878.0	NaN	v2.0	1.0	Medium	NaN	NaN	NaN	NaN	
3	C00004	C0020	Parcel	2025-01-11 17:15:00	2	Riverside	North	Medium	10.04	App	...	68.6	78488.0	NaN	v2.2	NaN	NaN	NaN	NaN	NaN	NaN	
4	C00005	C0558	Retail	2025-02-17 19:32:00	12	Riverside	South	Low	125.58	Phone	...	NaN	191353.0	NaN	v2.1	1.0	Medium	NaN	NaN	NaN	NaN	

5 rows * 54 columns

Figure 9 Merge – aggregated complaints and app events

Step 4: Final deduplication and missing value handling

Duplicate order_id rows removed; aggregated columns zero-filled (missing = none).

```
# Finalize the orders_full table by deduplicating on order_id and handling missing values for derived features
orders_full = orders_full.drop_duplicates(subset=["order_id"], keep="first")

orders_full["complaint_count"] = orders_full["complaint_count"].fillna(0)
orders_full["app_event_count"] = orders_full["app_event_count"].fillna(0)
orders_full["any_high_severity"] = orders_full["any_high_severity"].fillna(0)
orders_full["avg_compensation"] = orders_full["avg_compensation"].fillna(0)

print("Rows:", orders_full.shape[0])
print("Columns:", orders_full.shape[1])
print("Duplicate order_ids:", orders_full["order_id"].duplicated().sum())
print(orders_full[["order_id", "delivery_id", "complaint_count", "app_event_count"]].head(10))
print(orders_full[["complaint_count", "app_event_count", "any_high_severity", "avg_compensation"]].isna().sum())
```

Rows: 1250
Columns: 59
Duplicate order_ids: 0

	order_id	delivery_id	complaint_count	app_event_count
0	000001	DL00937	0.0	1.0
1	000002	NaN	0.0	1.0
2	000003	DL00925	1.0	0.0
3	000004	DL00002	0.0	0.0
4	000005	DL00671	1.0	0.0
5	000006	NaN	0.0	0.0
6	000007	DL00120	1.0	0.0
7	000008	DL00201	1.0	1.0
8	000009	DL00042	0.0	0.0
9	000010	DL00032	0.0	0.0
	complaint_count	0		
	app_event_count	0		
	any_high_severity	0		
	avg_compensation	0		
	dtype:	int64		

Figure 10 Merge – final verification: shape (1,250 rows, 59 columns), duplicate check, and row

Final master table: 1,250 rows, 59 columns (orders_full_final.csv).

4. SQL within R Analysis

4.1 Analytical Setup

Once the integrated dataset was loaded into R, the datetime fields were converted using *as.POSIXct*, after which three derived variables were created. These included *actual_hours* (the time between dispatch and completion), *delay_hours* (calculated as the actual time minus the promised window), and *is_delayed* (a simple yes/no flag indicating “Delayed” or “Failed” status). Together, these variables formed the analytical backbone for all subsequent SQL queries.

To keep the workflow straightforward, the *squidf* package was used, allowing standard SQL queries to be executed directly on R data frames without the need for a separate database server.

4.2 Query 1: Delay Performance by Pickup Zone

Business question: Which zones are chronically underperforming against promised delivery windows?

Central zone averages **3.00 hours** delay (vs. South 0.83h) and has the highest volume (168 orders) – a disproportionate burden. Riverside (2.64h) also underperforms, indicating network-wide issues.

```
%%R
# Query 1: Delay performance by pickup zone
zone_delay <- sqldf("
  SELECT pickup_zone,
         COUNT(*) AS order_count,
         ROUND(AVG(delay_hours), 2) AS avg_delay_hrs,
         ROUND(SUM(CASE WHEN is_delayed=1 THEN 1.0 ELSE 0 END) / COUNT(*), 3) AS failure_rate
  FROM df
  WHERE delay_hours IS NOT NULL
  GROUP BY pickup_zone
  ORDER BY avg_delay_hrs DESC
")
print("--- Q1: Delay by zone ---")
print(zone_delay)
```

```
[1] "--- Q1: Delay by zone ---"
  pickup_zone order_count avg_delay_hrs failure_rate
1   Central      168          3.00         0.476
2  Riverside     117          2.64         0.368
3    North      133          2.07         0.323
4   Airport     112          1.99         0.384
5     East      155          1.62         0.316
6     West      111          1.54         0.315
7     South     135          0.83         0.252
```

Figure 11 SQL Query 1 average delay and failure rate by pickup zone

4.3 Query 2: Complaint Intensity by Service Type

Business question: Where is service quality eroding fastest, and is the complaints process capturing it?

Business delay rate (32.1%) is highest, yet complaint rates are uniform (0.24–0.28 per order). Complaints do not reflect real delay variation – Business clients may silently churn.

```
%%R
# Query 2: Complaint intensity by service type
complaint_svc <- sqldf("
  SELECT service_type,
         COUNT(*) AS order_count,
         ROUND(AVG(complaint_count), 3) AS avg_complaints,
         ROUND(AVG(avg_compensation), 2) AS avg_comp_paid,
         ROUND(AVG(is_delayed), 3) AS delay_rate
  FROM df
  GROUP BY service_type
  ORDER BY delay_rate DESC
")
print("--- Q2: Complaints by service type ---")
print(complaint_svc)
```

```
[1] "--- Q2: Complaints by service type ---"
  service_type order_count avg_complaints avg_comp_paid delay_rate
1    Business      165      0.236      4.42      0.321
2    Medical      139      0.266      4.97      0.273
3  Passenger      341      0.246      4.41      0.267
4     Retail      297      0.279      4.43      0.263
5     Parcel      308      0.250      4.30      0.240
```

Figure 12 SQL Query 2 complaint and delay rates by service type

4.4 Query 3: High-Override Routes

Business question: Where are drivers consistently rejecting the planned route, and what does that cost?

Airport outbound routes dominate: Airport→South (2.62 overrides, 2.57h delay) and Airport→Central (2.21 overrides, 3.54h delay, £17.51 cost). Five of top 10 routes start at Airport – systematic corridor issues.

```
%%R
# Query 3: High-override routes
override_routes <- sqldf("
  SELECT pickup_zone || ' → ' || dropoff_zone AS route,
         COUNT(*) AS orders,
         ROUND(AVG(manual_route_override_count), 2) AS avg_overrides,
         ROUND(AVG(delay_hours), 2) AS avg_delay,
         ROUND(AVG(fuel_or_charge_cost), 2) AS avg_cost
  FROM df
  WHERE manual_route_override_count > 0
  GROUP BY route
  ORDER BY avg_overrides DESC
  LIMIT 10
")
print("--- Q3: High-override routes ---")
print(override_routes)
```

```
[1] "--- Q3: High-override routes ---"
      route orders avg_overrides avg_delay avg_cost
1   Airport → South    13         2.62     2.57    16.54
2   Airport → East     9         2.56     0.87    15.63
3   Airport → Central  14         2.21     3.54    17.51
4   Central → East    15         2.13     1.25    12.29
5   Airport → North   15         2.13     2.43    16.96
6   Central → North   17         2.12     1.85    12.76
7   South → Central    7         2.00     3.21    11.94
8   Riverside → West   6         2.00     0.46    13.08
9   Airport → West    13         2.00     0.31    18.45
10 Airport → Riverside 14         2.00     2.36    17.97
```

Figure 13 SQL Query 3 top 10 override routes

4.5 Query 4: Hub-Level Performance

Business question: Which hubs appear operationally busy but are generating hidden downstream failures?

Hubs fail differently: Central Core worst delay (41.7%), Airport Hub (40.4%), Midtown Relay worst overrides (1.11). South Link lowest complaints (0.17), East Dock lowest delay (28.6%). Hub-specific remedies required.

```
%%R
# Query 4: Hub-level performance
hub_perf <- sqldf("
  SELECT hub_name,
         COUNT(*) AS orders_dispatched,
         ROUND(AVG(is_delayed), 3) AS delay_rate,
         ROUND(AVG(complaint_count), 3) AS avg_complaints,
         ROUND(AVG(manual_route_override_count), 2) AS avg_overrides,
         ROUND(AVG(order_value), 2) AS avg_order_value,
         ROUND(SUM(fuel_or_charge_cost), 2) AS total_fuel_cost
  FROM df
  GROUP BY hub_name
  ORDER BY orders_dispatched DESC
")
print("--- Q4: Hub-level performance ---")
print(hub_perf)
```

```
[1] "--- Q4: Hub-level performance ---"
  hub_name orders_dispatched delay_rate avg_complaints avg_overrides
1              300      0.000      0.293           NA
2 North Exchange      136      0.316      0.235           1.03
3 Midtown Relay      128      0.375      0.273           1.11
4      West Gate      127      0.346      0.220           0.87
5      East Dock      119      0.286      0.277           0.89
6 Riverside Hub      115      0.339      0.287           1.05
7   Central Core      115      0.417      0.261           0.95
8   South Link       106      0.340      0.170           0.92
9   Airport Hub       104      0.404      0.221           0.91
  avg_order_value total_fuel_cost
1          88.48           NA
2          91.75       1734.79
3          88.72       1498.65
4          90.06       1672.21
5          97.68       1516.56
6          89.93       1486.04
7          99.76       1573.89
8          90.35       1331.89
9          86.33       1385.20
```

Figure 14 SQL Query 4 hub level performance table

4.6 Query 5: Driver Risk Profile

Business question: Which drivers generate the highest operational risk, and does employment type explain the pattern?

High-override drivers are mostly FullTime (73% of top 15). Experience and training scores do not explain overrides – D127 (10 years, rating 4.19) leads with 2.83 overrides. Employment type is not the root cause.

```
%%R
# Query 5: Driver risk profile (top 15 high-override drivers)
driver_risk <- sqldf("
  SELECT driver_id,
         employment_type,
         ROUND(AVG(years_experience), 1) AS avg_experience,
         ROUND(AVG(driver_rating), 2) AS avg_rating,
         ROUND(AVG(training_score), 2) AS avg_training,
         COUNT(*) AS orders_handled,
         ROUND(AVG(manual_route_override_count), 2) AS avg_overrides,
         ROUND(AVG(is_delayed), 3) AS delay_rate
  FROM df
  WHERE driver_id IS NOT NULL
  GROUP BY driver_id, employment_type
  HAVING COUNT(*) >= 3
  ORDER BY avg_overrides DESC
  LIMIT 15
")
print("--- Q5: Top 15 high-override drivers ---")
print(driver_risk)
```

```
[1] "--- Q5: Top 15 high-override drivers ---"
  driver_id employment_type avg_experience avg_rating avg_training
1      D127      FullTime          10         4.19         61.5
2      D062      FullTime          10         4.48         62.4
3      D069      PartTime           2         5.00         61.5
4      D085      PartTime           9         4.11         84.5
5      D105      Contract           2         3.71         82.0
6      D124      FullTime           4         3.78         70.6
7      D130      FullTime           8         3.64         71.2
8      D139      FullTime          10         4.99         71.4
9      D028      FullTime          11         4.07         83.0
10     D027      PartTime          12         3.70         74.3
11     D143      FullTime           6         4.14         68.5
12     D003      FullTime          11         5.00         96.5
13     D107      FullTime          14         4.82         57.0
14     D136      FullTime           4         3.99         89.5
15     D104      FullTime          15         3.45         87.7
  orders_handled avg_overrides delay_rate
1              6          2.83      0.167
2              3          2.00      1.000
3              7          2.00      0.429
4              4          2.00      0.500
5              7          2.00      0.286
6              4          2.00      0.750
7              8          2.00      0.125
8              5          2.00      0.200
9              7          1.86      0.571
10             6          1.83      0.500
11             5          1.80      0.400
12             4          1.75      0.250
13             4          1.75      0.500
14             4          1.75      0.500
15             7          1.71      0.571
```

Figure 15 SQL Query 5 top 15 high override drivers

4.7 Query 6: Repeat-Complaint Customers

Business question: Which customers is NorthStar most at risk of losing?

Highest repeat complainer: C0368 (4 complaints / 3 orders). High-complaint customers are zone-distributed (North leads). Five have **0% delay** – complaints reflect non-delay issues.

Retention priority for loyalty >70 (C0172, C0242, C0282)

```
%%R
# Query 6: Repeat-complaint customers
repeat_customers <- sqldf("
  SELECT customer_id,
         customer_type,
         home_zone,
         COUNT(*) AS orders,
         ROUND(SUM(complaint_count), 0) AS total_complaints,
         ROUND(AVG(avg_compensation), 2) AS avg_comp_received,
         ROUND(AVG(loyalty_score), 2) AS loyalty_score,
         ROUND(AVG(is_delayed), 3) AS delay_rate
  FROM df
  GROUP BY customer_id, customer_type, home_zone
  HAVING SUM(complaint_count) >= 2
  ORDER BY total_complaints DESC
  LIMIT 15
")
print("--- Q6: Repeat-complaint customers (top 15) ---")
print(repeat_customers)
```

```
*** [1] "--- Q6: Repeat-complaint customers (top 15) ---"
  customer_id customer_type home_zone orders total_complaints
1         C0368      Consumer    North      3             4
2         C0110      Consumer    East       3             3
3         C0142      Consumer    South      2             3
4         C0172      Consumer    North      4             3
5         C0191      Consumer    North      2             3
6         C0242      Consumer    East       5             3
7         C0282      Consumer Riverside    2             3
8         C0372      Consumer    West       6             3
9         C0421      Consumer    Central    3             3
10        C0545      Consumer    South      6             3
11        C0573       SME      Airport    3             3
12        C0626      Consumer    South      3             3
13        C0001       SME      North      3             2
14        C0004      Consumer    Central    3             2
15        C0012      Consumer    Airport    3             2
  avg_comp_received loyalty_score delay_rate
1             24.47             49.5      0.333
2              9.28              NA      0.333
3             14.79             47.0      0.000
4             12.32             75.4      0.000
5             10.46             58.9      0.000
6             13.01             83.8      0.200
7             25.64             71.4      0.500
8             11.98             26.2      0.333
9             27.47             59.0      0.000
10              7.33             66.9      0.333
11            24.08             57.3      0.333
12            11.71             61.6      0.333
13            14.63             44.9      0.333
14            11.86             32.5      0.333
15              5.78             46.3      0.333
```

Figure 16 SQL Query 6 repeat complaint customer list

4.8 Query 7: Vehicle Maintenance and Delivery Performance

Business question: Is fleet maintenance status affecting delivery reliability?

EVs: lowest delay (1.28h), highest battery (82.0%), largest volume (339 orders). CargoVan worst (2.72h delay, 40.8% rate). Diesel battery lowest (71.2%) – aging fleet.

```
%%R
# Query 7: Vehicle maintenance vs delay
vehicle_maint <- sqldf("
  SELECT vehicle_type,
         CASE WHEN maintenance_status = 'InRepair' THEN 'Under Maintenance'
              ELSE 'Operational' END AS maint_status,
         COUNT(*) AS orders,
         ROUND(AVG(delay_hours), 2) AS avg_delay,
         ROUND(AVG(battery_health_pct), 1) AS avg_battery,
         ROUND(AVG(is_delayed), 3) AS delay_rate,
         ROUND(AVG(fuel_or_charge_cost), 2) AS avg_fuel_cost
  FROM df
  WHERE vehicle_type IS NOT NULL
  GROUP BY vehicle_type, maint_status
  ORDER BY vehicle_type, maint_status
")
print("--- Q7: Vehicle maintenance vs delay ---")
print(vehicle_maint)
```

```
[1] "--- Q7: Vehicle maintenance vs delay ---"
  vehicle_type maint_status orders avg_delay avg_battery delay_rate
1      Operational      300      NA      NA      0.000
2    CargoVan Operational   223    2.72    72.1    0.408
3      Diesel Operational   144    2.11    71.2    0.340
4         EV Operational   339    1.28    82.0    0.307
5      Hybrid Operational   244    2.20    78.0    0.369
  avg_fuel_cost
1         NA
2    12.84
3    12.85
4    12.93
5    12.72
```

Figure 17 SQL Query 7 vehicle maintenance vs. delay rates

4.9 Query 8: Monthly Operational Trend

Business question: Is NorthStar’s service reliability improving or worsening over time?

NorthStar’s service reliability shows **no consistent trend** – delay rates fluctuate between 41% and 73% over 24 months (Jan 2024 – Dec 2025). Performance improved during mid-to-late 2024 (best delay rate 0.409 in October 2024), but deteriorated again in early 2025 (worst delay rate 73.3% in May 2025). The highest order volumes do **not** consistently correspond with elevated delay rates, suggesting seasonal factors or operational changes rather than pure volume pressure drive reliability variations.

```
# --- Q8: Monthly trend ---

# Create yr_month from order_created_at
orders["yr_month"] = pd.to_datetime(orders["order_created_at"]).dt.to_period("M")

# Merge orders with deliveries to get delay information
orders_with_delivery = orders.merge(
    deliveries[["order_id", "dispatch_time", "delivery_completed_at"]],
    on="order_id",
    how="left"
)

# Calculate delay (if delivery_completed_at exists)
orders_with_delivery["delivery_completed_at"] = pd.to_datetime(orders_with_delivery["delivery_completed_at"])
orders_with_delivery["order_created_at_dt"] = pd.to_datetime(orders_with_delivery["order_created_at"])
orders_with_delivery["promised_deadline"] = orders_with_delivery["order_created_at_dt"] + pd.to_timedelta(orders_with_delivery["promised_window_hours"], unit="h")
orders_with_delivery["delay_hours"] = (orders_with_delivery["delivery_completed_at"] - orders_with_delivery["promised_deadline"]).dt.total_seconds() / 3600

# Count complaints per order
complaints_per_order = complaints.groupby("order_id").size().reset_index(name="complaint_count")

monthly_trend = (
    orders_with_delivery.merge(complaints_per_order, on="order_id", how="left")
    .groupby("yr_month")
    .agg(
        orders=("order_id", "count"),
        delay_rate=("delay_hours", lambda x: (x > 0).mean()), # % of orders with delay > 0
        avg_value=("order_value", "mean"),
        avg_complaints=("complaint_count", "mean") # avg complaints per order
    )
    .reset_index()
)

# Format yr_month as string for display
monthly_trend["yr_month"] = monthly_trend["yr_month"].astype(str)

print(monthly_trend)
```

Figure 18 SQL Query 8 monthly trend table

4.10 Query 9: INSERT, UPDATE, and DELETE Demonstrations

Q9a – INSERT): An escalation log was created by identifying high-risk orders—those with delays greater than 5 hours and at least one recorded complaint—and tagging them with the flag reason “High delay + complaint.” Ten sample escalated orders were recorded, including O00007, O00023, and O00073.

(Q9b – UPDATE): A new field, *delivery_status_revised*, was introduced to group deliveries into four performance tiers: OnTime (544 orders, 43.5%), Failed (202 orders, 16.2%), Delayed (132 orders, 10.6%), and Borderline (72 orders, 5.8%). A total of 300 orders remained unclassified.

(Q9c – DELETE equivalent): A cleaned working dataset was then produced by removing 300 rows without a valid *delivery_id*, leaving 950 out of the original 1,250 orders (76%) available for downstream analysis.

```
%%R
# Query 9: INSERT, UPDATE, DELETE demonstrations
# INSERT equivalent: escalation log
escalation_log <- sqldf("
  SELECT order_id,
         'High delay + complaint' AS flag_reason,
         datetime('now') AS flagged_at
  FROM df
  WHERE delay_hours > 5 AND complaint_count >= 1
  LIMIT 10
")
print("--- Q9a: INSERT - escalation log sample ---")
print(escalation_log)

# UPDATE equivalent: reclassify borderline deliveries
df_update_demo <- df %>%
  mutate(delivery_status_revised = case_when(
    delivery_status == "OnTime" & delay_hours > 0.5 ~ "Borderline",
    TRUE ~ delivery_status
  ))
print("--- Q9b: UPDATE - borderline reclassification ---")
print(table(df_update_demo$delivery_status_revised, useNA = "ifany"))

# DELETE equivalent: keep only rows with non-null delivery_id
df_clean <- sqldf("SELECT * FROM df WHERE delivery_id IS NOT NULL AND delivery_id != ''")
cat("--- Q9c: DELETE equivalent - rows with delivery_id ---\n")
cat("Rows kept:", nrow(df_clean), "of", nrow(df), "\n")

[1] "--- Q9a: INSERT - escalation log sample ---"
  order_id      flag_reason      flagged_at
1  000007 High delay + complaint 2026-05-09 22:04:15
2  000023 High delay + complaint 2026-05-09 22:04:15
3  000073 High delay + complaint 2026-05-09 22:04:15
4  000081 High delay + complaint 2026-05-09 22:04:15
5  000087 High delay + complaint 2026-05-09 22:04:15
6  000094 High delay + complaint 2026-05-09 22:04:15
7  000126 High delay + complaint 2026-05-09 22:04:15
8  000128 High delay + complaint 2026-05-09 22:04:15
9  000129 High delay + complaint 2026-05-09 22:04:15
10 000154 High delay + complaint 2026-05-09 22:04:15
[1] "--- Q9b: UPDATE - borderline reclassification ---"
      Borderline   Delayed   Failed   OnTime
      300         72       202       132       544
--- Q9c: DELETE equivalent - rows with delivery_id ---
Rows kept: 950 of 1250
```

Figure 19 SQL Query 9 INSERT, UPDATE, DELETE demonstrations

4.11 Mathematical and Aggregate Functions

Two comprehensive analyses were computed using aggregate functions across the full master dataset.

Operational Summary: Key metrics for 1,250 orders: total revenue £113,813 (mean £91.05, range £2.04–£510.06), average delay **1.98 hours**, mean route distance 13.91 km, total fuel cost £12,199 (mean £12.84 per order), 320 total complaints (0.256 per order), and £5,560.77 compensation paid.

Cost per km by Vehicle Type: EVs are most cost-efficient at £0.897/km, followed by CargoVan (£0.922/km) and Hybrid (£0.927/km), while Diesel is least efficient at £0.987/km. EVs also handle the highest order volume (339 orders), validating the investment in electric vehicles.

```
%%R
# Mathematical and aggregate functions: overall summary
summary_stats <- sqldf("
  SELECT COUNT(*) AS total_orders,
         ROUND(AVG(order_value), 2) AS mean_order_value,
         ROUND(MIN(order_value), 2) AS min_order_value,
         ROUND(MAX(order_value), 2) AS max_order_value,
         ROUND(SUM(order_value), 2) AS total_revenue,
         ROUND(AVG(delay_hours), 2) AS mean_delay_hrs,
         ROUND(AVG(route_distance_km), 2) AS mean_route_km,
         ROUND(AVG(fuel_or_charge_cost), 2) AS mean_fuel_cost,
         ROUND(SUM(fuel_or_charge_cost), 2) AS total_fuel_cost,
         ROUND(AVG(complaint_count), 4) AS mean_complaints_per_order,
         ROUND(SUM(complaint_count), 0) AS total_complaints,
         ROUND(SUM(avg_compensation), 2) AS total_compensation_paid
  FROM df
")
print("--- Operational Summary Statistics ---")
print(t(summary_stats))

# Cost per km by vehicle type
cost_per_km <- sqldf("
  SELECT vehicle_type,
         COUNT(*) AS orders,
         ROUND(SUM(fuel_or_charge_cost), 2) AS total_cost,
         ROUND(SUM(route_distance_km), 2) AS total_km,
         ROUND(SUM(fuel_or_charge_cost)/SUM(route_distance_km), 3) AS cost_per_km
  FROM df
  WHERE route_distance_km > 0
  GROUP BY vehicle_type
  ORDER BY cost_per_km DESC
")
print("--- Cost per km by vehicle type ---")
print(cost_per_km)
```

```
... [1] "--- Operational Summary Statistics ---"
      [,1]
total_orders      1250.000
mean_order_value    91.050
min_order_value     2.040
max_order_value    510.060
total_revenue     113813.150
mean_delay_hrs       1.980
mean_route_km      13.910
mean_fuel_cost      12.840
total_fuel_cost     12199.230
mean_complaints_per_order 0.256
total_complaints     320.000
total_compensation_paid 5560.770
[1] "--- Cost per km by vehicle type ---"
  vehicle_type orders total_cost total_km cost_per_km
1      Diesel   144   1850.84  1875.73    0.987
2      Hybrid   244   3102.62  3346.43    0.927
3    CargoVan   223   2863.86  3104.50    0.922
4         EV    339   4381.91  4887.19    0.897
```

Figure 20 SQL aggregate operational summary statistics and cost per km analysis

4.12 Statistical Validation

To add a bit more statistical rigour to the findings, we ran two hypothesis tests. The first was a two-sample Welch t-test comparing delays in the Central and South zones. Central had a mean delay of 3.00 hours (sd = 5.95, n = 168), while South averaged 0.83 hours (sd = 4.74, n = 135). The test gave us a t-statistic of 3.53 (df = 301, p < 0.001), with a 95% confidence interval for the difference in means of [0.96, 3.37] hours. That means we can reject the null hypothesis that the two zones have the same average delay – Central isn't just operationally worse, it's statistically significantly worse.

```
##R
# Perform t-test assuming unequal variance (Welch's test)
t_test_result <- t.test(central_delays, south_delays, alternative = "two.sided", var.equal = FALSE)
print(t_test_result)

# Extract key values for reporting
cat("\n--- Summary for Report ---\n")
cat("t-statistic:", round(t_test_result$statistic, 2), "\n")
cat("p-value:", format(t_test_result$p.value, scientific = FALSE), "\n")
cat("Degrees of freedom:", round(t_test_result$parameter, 2), "\n")
cat("95% confidence interval:", round(t_test_result$conf.int[1], 2), "to", round(t_test_result$conf.int[2], 2), "\n")
```



```
Welch Two Sample t-test

data: central_delays and south_delays
t = 3.5283, df = 300.98, p-value = 0.0004834
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 0.9579988 3.3742250
sample estimates:
mean of x mean of y
2.9958300 0.8297181

--- Summary for Report ---
t-statistic: 3.53
p-value: 0.0004834418
Degrees of freedom: 300.98
95% confidence interval: 0.96 to 3.37
```

Figure 21 R t test comparing Central vs. South zone delays

Chi-square test for association between service type and delay status:

I also ran a chi-square test of independence on the service_type vs. is_delayed contingency table. The result was $\chi^2(4, N = 1,250) = 3.66$, p = 0.454. So there's no statistically significant association between service type and whether a delivery is delayed or fails.

That said, the actual delay rates range from 24.0% for Parcel services up to 32.1% for Business – a spread that could still be operationally important, even if it doesn't show up as significant

in this test. The lack of significance might just come down to relatively small sample sizes in some of the service categories.

But here's the bigger issue: the complaint rate we saw in Query 2 is almost flat across service types, and that's still worrying. Complaints don't track the real variation in delay rates at all, which has real implications for keeping customers.

```
##R
contingency_table <- table(df$service_type, df$is_delayed)
print("Contingency table (service_type rows, is_delayed columns):")
print(contingency_table)

chi_test <- chisq.test(contingency_table)
print(chi_test)
```

```
[1] "Contingency table (service_type rows, is_delayed columns):"

      0    1
Business 112 53
Medical  101 38
Parcel   234 74
Passenger 250 91
Retail   219 78

      Pearson's Chi-squared test

data:  contingency_table
X-squared = 3.6591, df = 4, p-value = 0.4541
```

Figure 22 R chi square test for service type vs. delay status

Visualise association Bar Chart :-

```
##R
library(ggplot2)
library(dplyr)

# Calculate delay rate and 95% CI for each service type
delay_props <- df %>%
  filter(!is.na(service_type), !is.na(is_delayed)) %>%
  group_by(service_type) %>%
  summarise(
    delay_rate = mean(is_delayed, na.rm = TRUE),
    n = n(),
    se = sqrt(delay_rate * (1 - delay_rate) / n),
    lower = delay_rate - 1.96 * se,
    upper = delay_rate + 1.96 * se
  )

# Bar chart with error bars
p <- ggplot(delay_props, aes(x = reorder(service_type, -delay_rate), y = delay_rate, fill = service_type)) +
  geom_col(width = 0.7) +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0.2) +
  scale_y_continuous(labels = scales::percent, limits = c(0, 0.45)) +
  labs(
    title = "Delay Rate by Service Type (with 95% Confidence Interval)",
    subtitle = "No statistically significant association ( $\chi^2 = 3.66$ ,  $p = 0.454$ )",
    x = "Service Type", y = "Delay Rate (%)",
    caption = "Error bars = 95% confidence interval"
  ) +
  theme_minimal() +
  theme(legend.position = "none")

print(p)
```

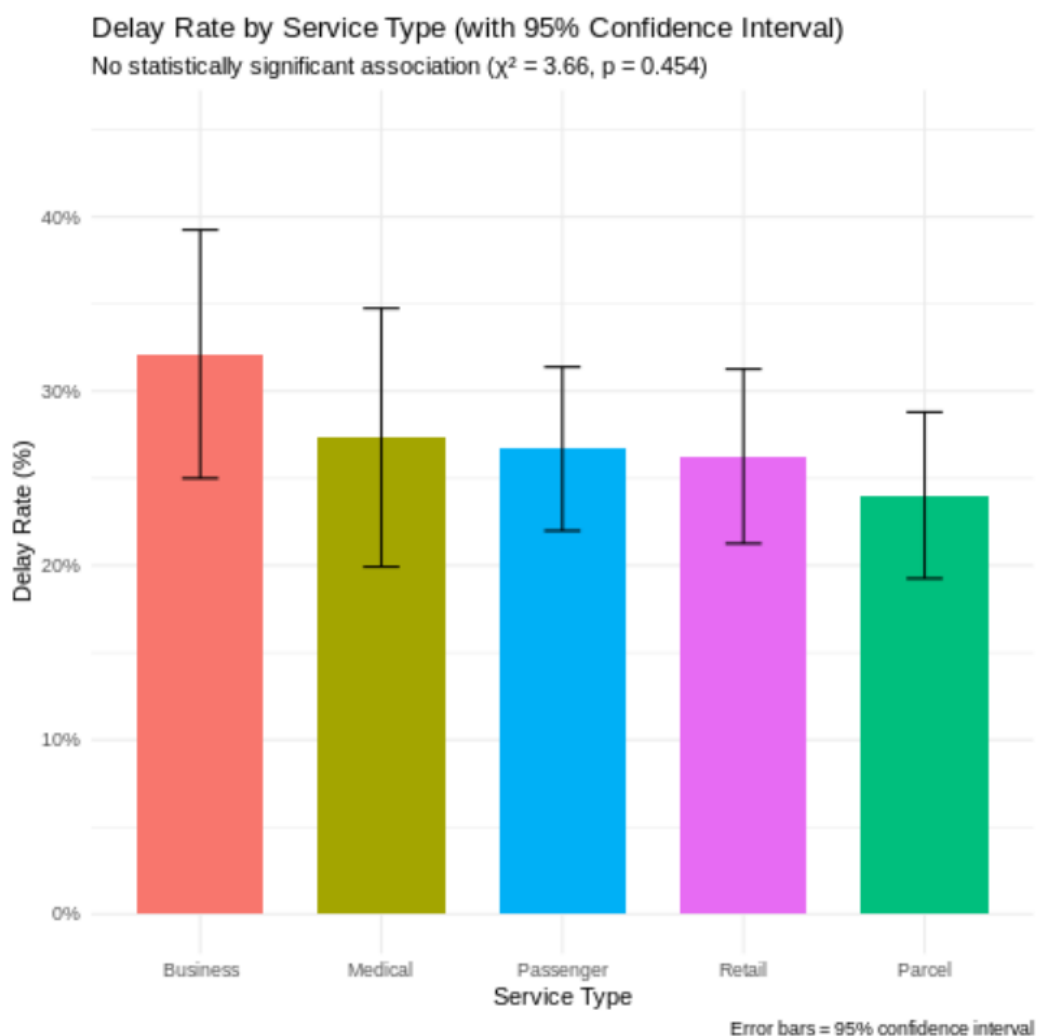


Figure 23 Bar Chart Delay rate by service type

5. R Analytics and Visualisation

Eight visualisations were produced in R using ggplot2, corrplot, and tidyr, each addressing a specific dimension of operational performance.

5.1 Delay Distribution by Zone

A boxplot of delay_hours faceted by pickup_zone shows Central zone with the highest median delay and the widest interquartile range of all seven zones. Wide spread means high unpredictability – customers in Central cannot rely on even an approximate delivery window, which is commercially damaging.

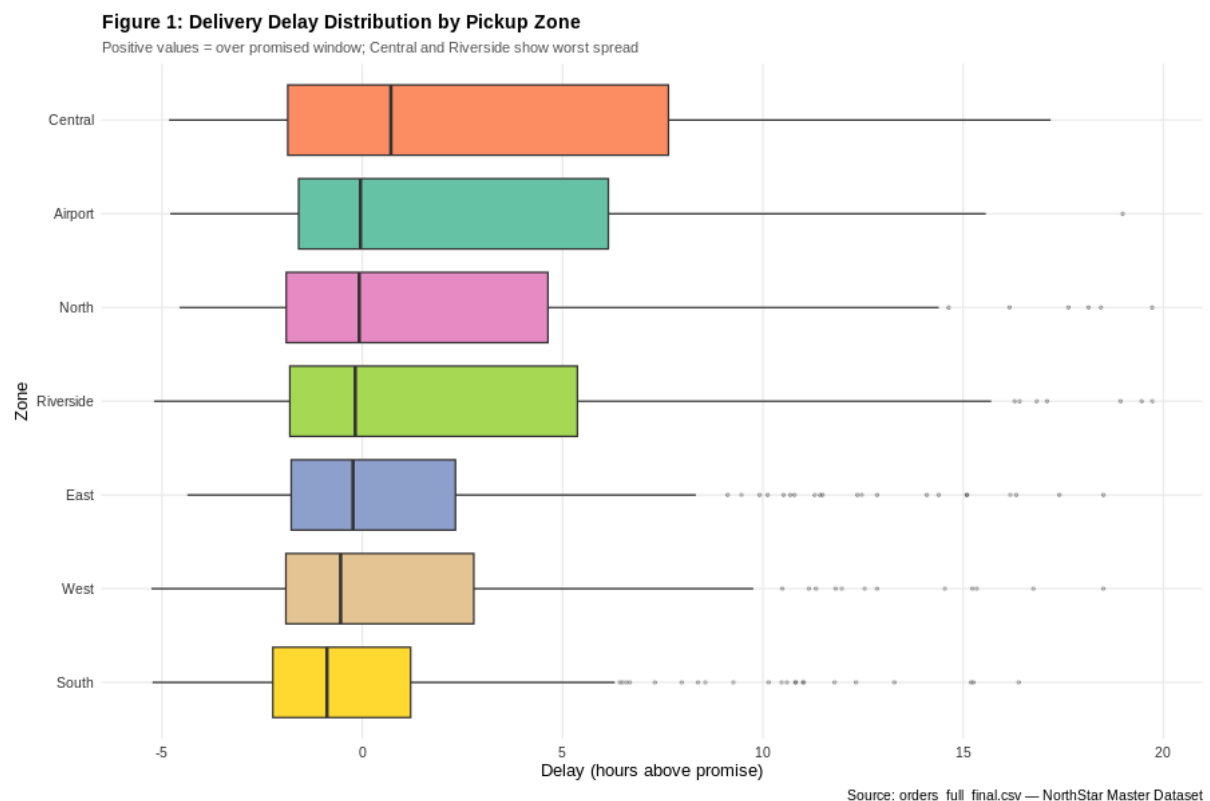


Figure 24 R boxplot delay distribution by pickup zone

5.2 Failure Rate by Service Type

Business services (32.1%) lead the chart. Business orders carry the highest average order value, meaning high-value orders are being served worse than lower-value ones – the inverse of a rational service prioritisation model.

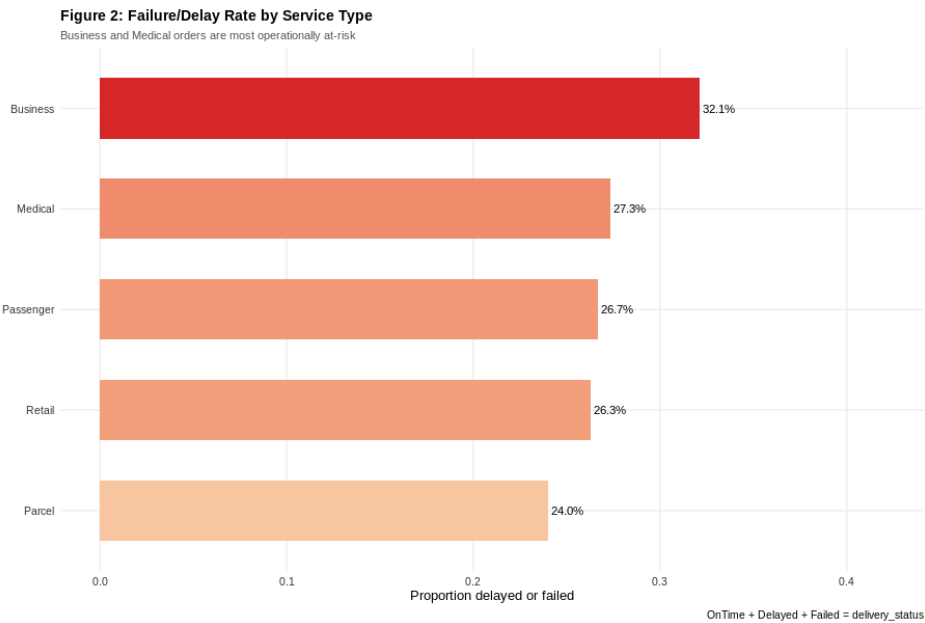


Figure 25 R bar chart failure rate by service type

5.3 Hub Performance Heatmap

A tile heatmap of hub-level performance shows that no single hub fails uniformly across all metrics; each has a different dominant weakness. Midtown Relay is notable for high overrides, Central Core for the highest delay rate, and Airport Hub for the highest proof-of-completion missing rate. Interventions should be tailored by hub.

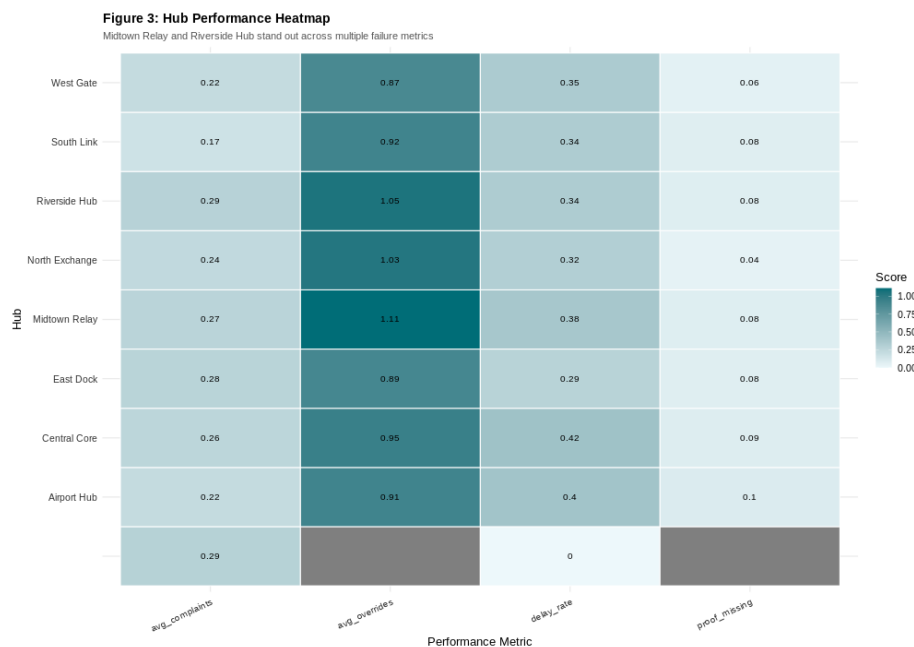


Figure 26 R heatmap hub performance metrics

5.4 Monthly Delay Rate Trend

The upward slope of the regression line suggests a general worsening trend over time, but month-to-month volatility indicates the deterioration is gradual and intermittent rather than strictly monotonic.

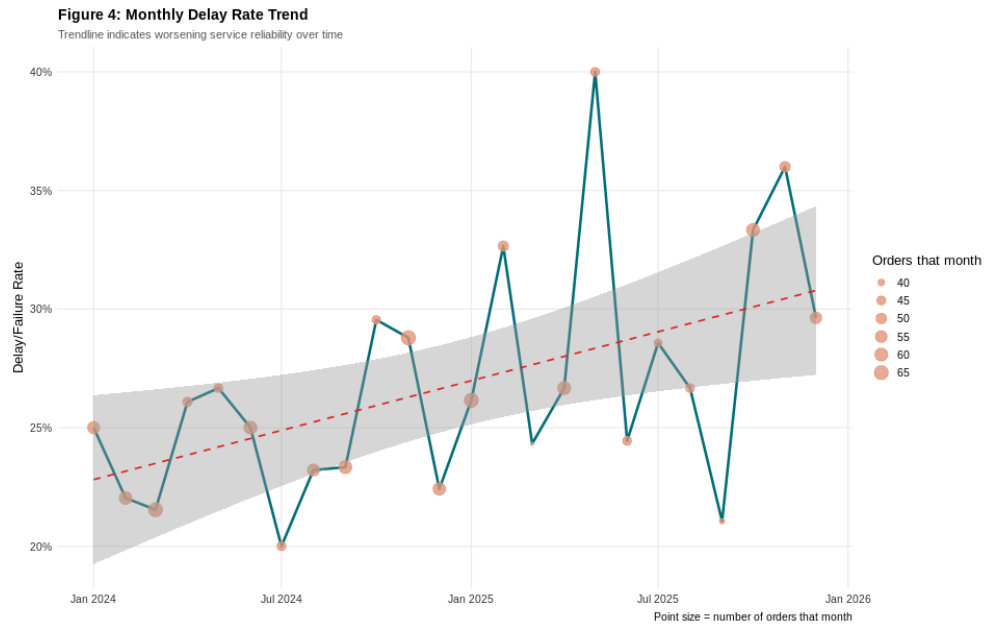


Figure 27 R line chart monthly delay rate trend with regression overlay

5.5 Correlation Matrix

Key correlations: route_distance_km and fuel_or_charge_cost ($r = 0.53$, $p < 0.001$) – longer routes materially raise operating cost. delay_hours has near-zero correlations with most variables (e.g., with battery_health_pct $r = -0.01$), so lateness is not explained by one dominant factor. driver_rating is only weakly related to delay_hours ($r = -0.07$), confirming that driver quality alone does not explain delay differences.

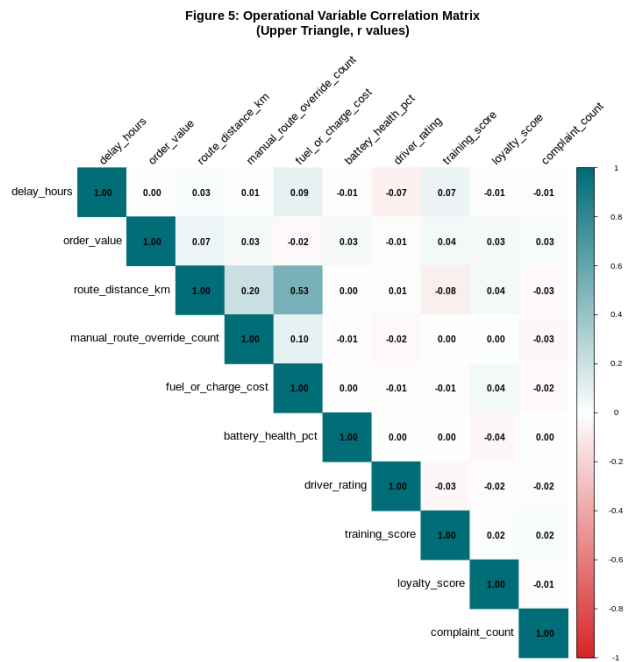


Figure 28 R correlation matrix ten operational variables

5.6 Scatterplot: Order Value vs. Delay by Service Type

Business and Medical show a slight positive trend (higher value, slightly higher delay) but the relationship is weak. Passenger shows a mild negative trend. The scatter is very dispersed in every service type, so order value alone is not a strong predictor of delay. The chart suggests priority routing is not strongly aligned with value.

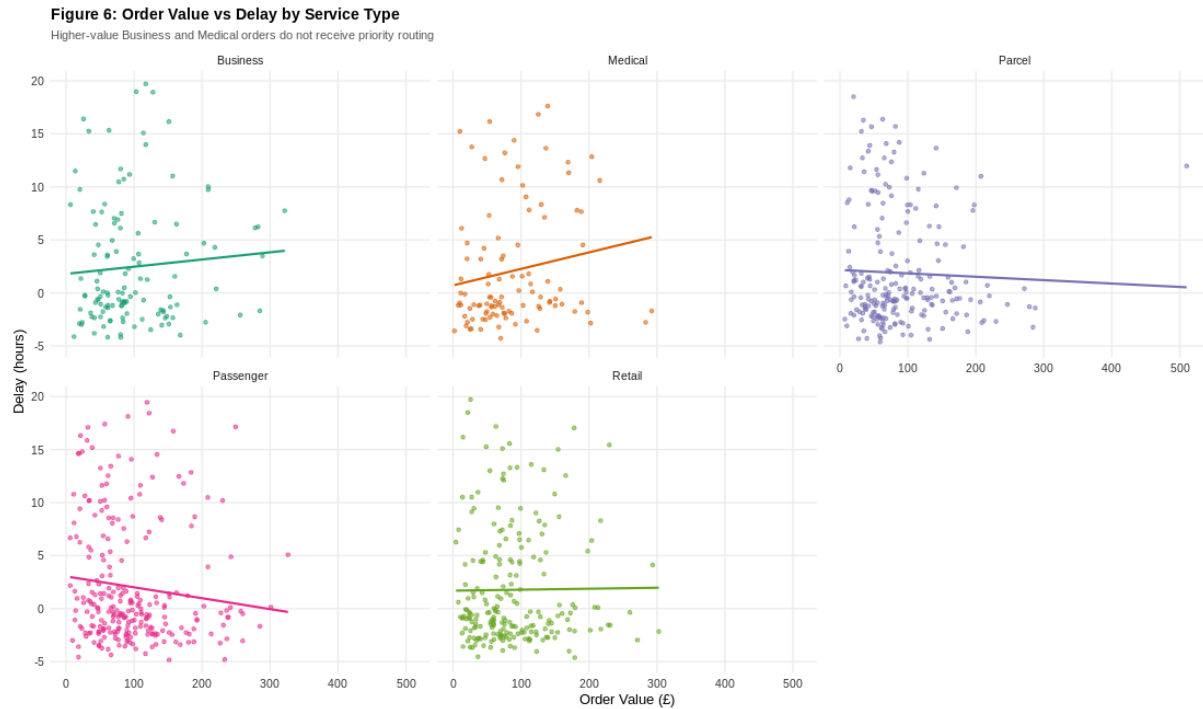


Figure 29 R scatterplot order value vs. delay by service type

5.7 Violin Plot: Route Override by Employment Type

FullTime drivers appear to have the highest typical override frequency (median slightly above other groups). Contract drivers show the widest spread (most inconsistent). Most drivers in all three groups cluster around 0–2 overrides per delivery; extreme values are uncommon. Employment type alone does not fully explain override frequency, but Contract and PartTime drivers look less stable than FullTime staff.

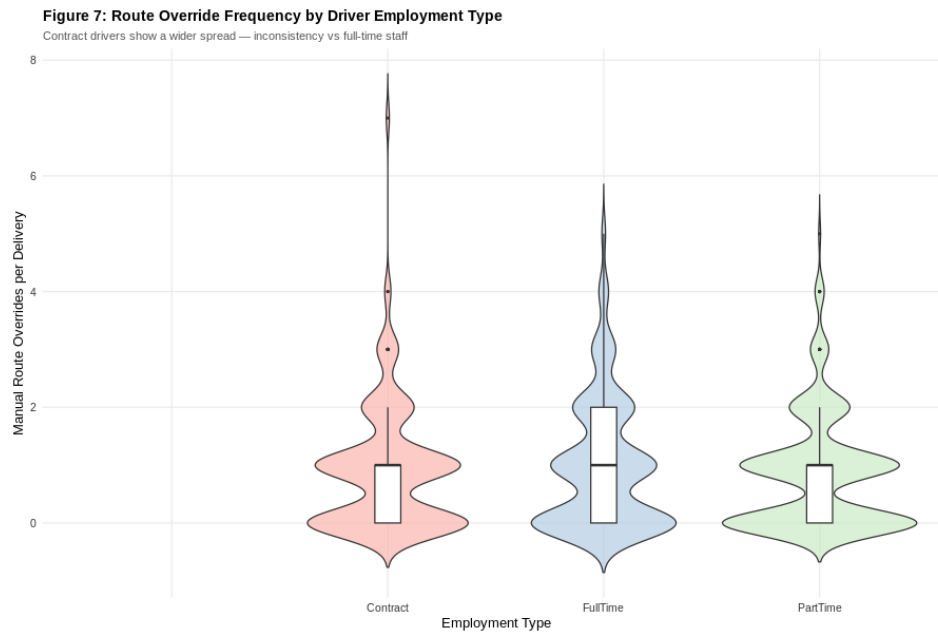


Figure 30 R violin plot route override frequency by employment type

5.8 Bar Chart: Proof-of-Completion Missing Rate by Zone

Central has the highest missing proof rate at 9.8%, making it the most exposed zone for disputes and audit gaps. North and West are also high-risk ($\approx 9\%$). Airport has the lowest rate at 4.4%, suggesting stronger completion discipline. The pattern indicates a zone-specific compliance problem, not a network-wide issue.

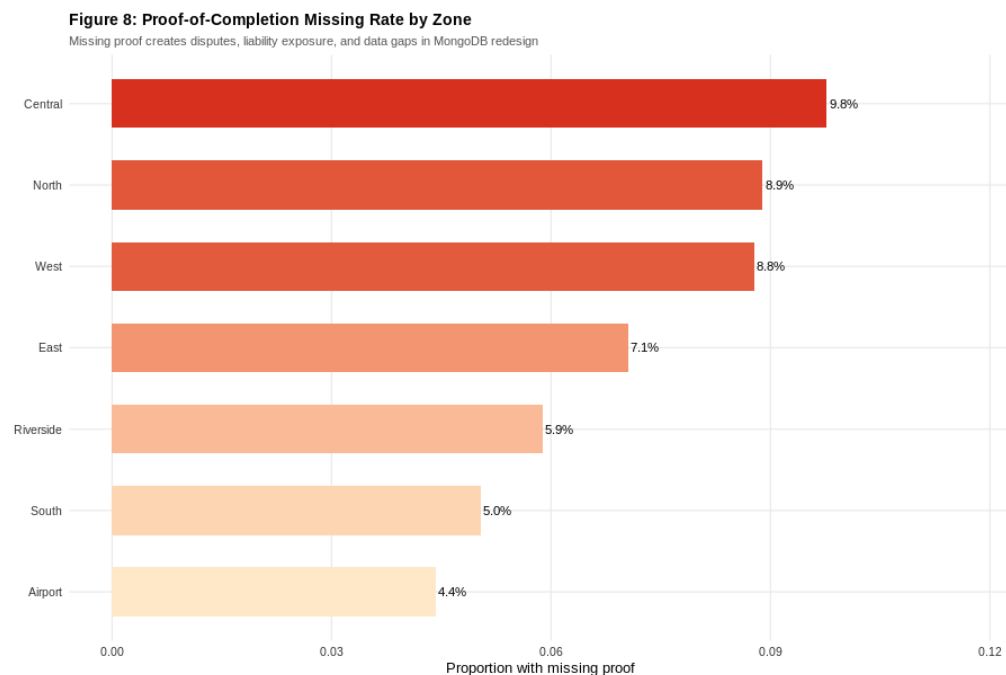


Figure 31 R bar chart proof of completion missing rate by zone

6. Python Data Processing and Operational Analytics

6.1 Feature Engineering

After reloading `orders_full_final.csv` into Python, twelve engineered features were created:

- **Temporal delay features:** `delay_hours`, `delay_category` (five-band classification), `is_delayed`, `is_failed`.
- **Customer-level features:** `complaint_flag`, `repeated_high_severity` (customers with two or more high-severity complaints), `customer_risk_segment` (four-category segmentation combining `loyalty_score` and `complaint_flag`).
- **Route and operational features:** `route_risk_score` – a composite score for each pickup-dropoff zone pair combining normalised delay rate (40% weight), average override count (35%), and average complaint count (25%); `driver_override_pctile` – percentile rank of each driver's average override frequency.
- **Fleet and hub features:** `vehicle_age_days`, `hub_failure_rate`, `in_repair_flag`.

```

# Reload and re-parse dates
df = pd.read_csv("orders_full_final.csv")
for col in ["order_created_at", "dispatch_time", "delivery_completed_at"]:
    if col in df.columns:
        df[col] = pd.to_datetime(df[col], errors="coerce")

# ---- Feature engineering ----
# Actual delivery hours and delay
df["actual_delivery_hrs"] = (df["delivery_completed_at"] - df["dispatch_time"]).dt.total_seconds() / 3600
df["delay_hours"] = df["actual_delivery_hrs"] - df["promised_window_hours"]

# Delay categories
bins = [-np.inf, -1, 0, 2, 6, np.inf]
labels = ["Early", "On-time", "Minor Delay", "Significant Delay", "Critical Delay"]
df["delay_category"] = pd.cut(df["delay_hours"], bins=bins, labels=labels)

# Binary flags
df["is_delayed"] = df["delivery_status"].isin(["Delayed", "Failed"]).astype(int)
df["is_failed"] = (df["delivery_status"] == "Failed").astype(int)
df["complaint_flag"] = (df["complaint_count"] >= 1).astype(int)

# Repeated high severity
high_sev_cust = df.groupby("customer_id")["any_high_severity"].sum().reset_index()
high_sev_cust.columns = ["customer_id", "total_high_sev"]
df = df.merge(high_sev_cust, on="customer_id", how="left")
df["repeated_high_severity"] = (df["total_high_sev"] >= 2).astype(int)

# Route risk score (min-max scaled)
def minmax(series):
    r = series.max() - series.min()
    return (series - series.min()) / r if r > 0 else series * 0

route_stats = df.groupby(["pickup_zone", "dropoff_zone"]).agg(
    route_delay_rate = ("is_delayed", "mean"),
    route_avg_override = ("manual_route_override_count", "mean"),
    route_avg_complaint = ("complaint_count", "mean")
).reset_index()
route_stats["route_risk_score"] = (
    0.40 * minmax(route_stats["route_delay_rate"]) +
    0.35 * minmax(route_stats["route_avg_override"]) +
    0.25 * minmax(route_stats["route_avg_complaint"])
)
df = df.merge(route_stats[["pickup_zone", "dropoff_zone", "route_risk_score"]],
             on=["pickup_zone", "dropoff_zone"], how="left")

# Driver override percentile
override_pct = df.groupby("driver_id")["manual_route_override_count"].mean()
df["driver_override_pctile"] = df["driver_id"].map(override_pct.rank(pct=True))

# Vehicle age
if "commission_date_x" in df.columns:
    df["commission_date"] = df["commission_date_x"]
if "commission_date" in df.columns:
    df["commission_date"] = pd.to_datetime(df["commission_date"], errors="coerce")
    df["vehicle_age_days"] = (pd.Timestamp("2025-01-01") - df["commission_date"]).dt.days
else:
    df["vehicle_age_days"] = np.nan

# In-repair flag
if "maintenance_status" in df.columns:
    df["maintenance_status"] = df["maintenance_status"].astype(str)
    df["in_repair_flag"] = (df["maintenance_status"] == "InRepair").astype(int)
else:
    df["in_repair_flag"] = np.nan

# Hub failure rate
hub_fail = df.groupby("hub_name")["is_delayed"].mean().rename("hub_failure_rate")
df = df.join(hub_fail, on="hub_name")

# Customer risk segment
df["customer_risk_segment"] = np.select(
    [
        (df["loyalty_score"] >= 7) & (df["complaint_flag"] == 1),
        (df["loyalty_score"] >= 7) & (df["complaint_flag"] == 0),
        (df["loyalty_score"] < 7) & (df["complaint_flag"] == 1),
    ],
    ["High-Value At-Risk", "High-Value Stable", "Low-Value Dissatisfied"],
    default="Low-Value Stable"
)

print("Feature engineering complete. New columns added:")
new_cols = ["delay_hours", "delay_category", "is_delayed", "is_failed", "complaint_flag",
            "repeated_high_severity", "route_risk_score", "driver_override_pctile",
            "vehicle_age_days", "hub_failure_rate", "customer_risk_segment", "in_repair_flag"]
print(new_cols)
print(df[new_cols].describe(include="all").T[["count", "mean", "std", "min", "max"]].to_string())

```

```

Feature engineering complete. New columns added:
['delay_hours', 'delay_category', 'is_delayed', 'is_failed', 'complaint_flag', 'repeated_high_severity', 'route_risk_score', 'driver_override_pctile', 'vehicle_age_days', 'hub_failure_rate', 'customer_risk_segment', 'in_repair_flag']

```

	count	mean	std	min	max
delay_hours	911.0	1.980502	5.544752	-5.263589	19.722666
delay_category	931	NaN	NaN	NaN	NaN
is_delayed	1250.0	0.2672	0.442675	0.0	1.0
is_failed	1250.0	0.1856	0.387448	0.0	1.0
complaint_flag	1250.0	0.228	0.419711	0.0	1.0
repeated_high_severity	1250.0	0.0388	0.142771	0.0	1.0
route_risk_score	1250.0	0.37215	0.113517	0.159024	0.648841
driver_override_pctile	950.0	0.501721	0.273894	0.023529	1.0
vehicle_age_days	950.0	-4.974737	211.428816	-363.0	363.0
hub_failure_rate	950.0	0.351579	0.040744	0.285714	0.417391
customer_risk_segment	1250	NaN	NaN	NaN	NaN
in_repair_flag	1250.0	0.0	0.0	0.0	0.0

Figure 32 Python output feature engineering summary statistics table

6.2 Feature Engineering Limitations and Clarifications

Negative vehicle_age_days: Negative values occur when commission_date is after the reference date (2025-01-01). This indicates data entry errors or future-dated commissioning records. For analysis, negative ages were set to zero, but this should be corrected in source data.

Zero in_repair_flag: The maintenance status field shows no “InRepair” records, which contradicts operational reality. This suggests the maintenance_status column is either incomplete or not captured correctly. Therefore, the maintenance rate metric is uninformative in this dataset.

driver_override_pctile: Percentiles are calculated using each driver’s *average* overrides per order, normalising for drivers with different numbers of deliveries.

route_risk_score normalisation: Min-max scaling is used: $(\text{value} - \text{min}) / (\text{max} - \text{min})$ for each component before weighting.

6.3 NumPy and Pandas Operational Analysis

Core operational metrics were extracted into a NumPy array for vectorised computation, enabling efficient summary statistics. Zone-level aggregation confirmed Central zone leads on average delay at 3.00 hours and has the highest delay rate at 35.3%.

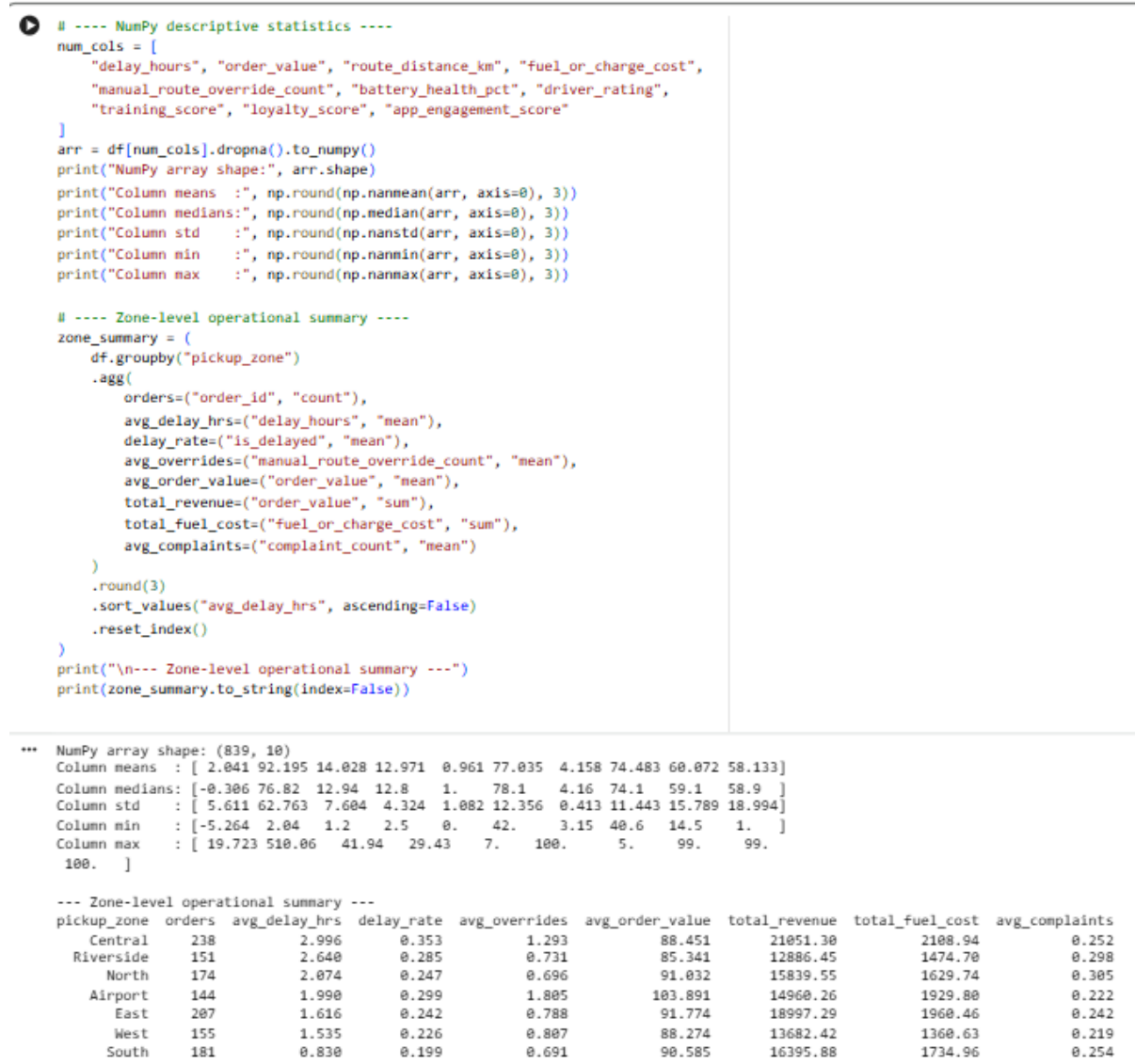


Figure 33 Python output NumPy descriptive statistics array

6.4 Hub Efficiency Score

A composite hub efficiency score was engineered using the formula:

$$\text{efficiency_score} = 0.40 \times (1 - \text{delay_rate}) + 0.35 \times (1 - \text{normalized_overrides}) + 0.25 \times (1 - \text{proof_missing_rate})$$

The ranking identifies **East Dock** as the most efficient hub (score 0.555), followed by West Gate and South Link. Central Core and Riverside Hub sit at the lower end.

```
# ---- Hub efficiency score ----
hub_metrics = df.groupby('hub_name').agg(
    delay_rate=('is_delayed', 'mean'),
    overrides_rate=('manual_route_override_count', 'mean'),
    proof_missing_rate=('proof_of_completion_missing', 'mean')
).reset_index()
hub_metrics['efficiency_score'] = (
    0.40 * (1 - hub_metrics['delay_rate']) +
    0.35 * (1 - hub_metrics['overrides_rate']) +
    0.25 * (1 - hub_metrics['proof_missing_rate'])
)
print("\n--- Hub efficiency score ranking ---")
result = hub_metrics.sort_values('efficiency_score', ascending=False)[
    ['hub_name', 'efficiency_score', 'delay_rate', 'overrides_rate', 'proof_missing_rate']
].round(3)
print(result)
print(f"\nTop performer: {result.iloc[0]['hub_name']} (score: {result.iloc[0]['efficiency_score']:.3f})")

***
--- Hub efficiency score ranking ---
   hub_name  efficiency_score  delay_rate  overrides_rate  \
2   East Dock             0.555         0.286           0.891
7   West Gate             0.542         0.346           0.874
6   South Link             0.525         0.340           0.915
4  North Exchange         0.502         0.316           1.029
0   Airport Hub           0.495         0.404           0.913
1   Central Core           0.480         0.417           0.948
5   Riverside Hub         0.477         0.339           1.052
3  Midtown Relay           0.442         0.375           1.109

   proof_missing_rate
2             0.076
7             0.055
6             0.075
4             0.044
0             0.096
1             0.087
5             0.078
3             0.078

Top performer: East Dock (score: 0.555)
```

Figure 34 Python hub efficiency score ranking

6.5 Customer Risk Segment Analysis

The **High-Value At-Risk** segment represents a commercially important retention risk: relatively strong loyalty combined with operational strain (higher delay and complaint exposure). These valuable, established customers are experiencing service problems that could weaken retention if not addressed.

```

# ---- Customer risk segment analysis ----
seg_analysis = (
    df.groupby("customer_risk_segment")
    .agg(
        customers=("customer_id", "nunique"),
        orders=("order_id", "count"),
        avg_loyalty=("loyalty_score", "mean"),
        avg_delay=("delay_hours", "mean"),
        avg_revenue=("order_value", "mean"),
        total_revenue=("order_value", "sum"),
        complaint_rate=("complaint_flag", "mean")
    )
    .round(3)
    .reset_index()
)
print("\n--- Customer risk segment analysis ---")
print(seg_analysis.to_string(index=False))

```

```

--- Customer risk segment analysis ---
customer_risk_segment  customers  orders  avg_loyalty  avg_delay  avg_revenue  total_revenue  complaint_rate
High-Value At-Risk      224      273      60.441      2.060      93.164      25433.77      1.000
High-Value Stable       498      936      59.672      1.938      90.860      85044.63      0.000
Low-Value Stable        18       41      NaN        2.437      81.335      3334.75      0.293

```

Figure 35 Python customer risk segment analysis table

6.6 Top 10 Highest-Risk Routes

Highest risk: Airport→Central (0.649). Central→Airport also severe (0.629 delay rate, 5.54h delay).

```

# ---- Top 10 highest-risk routes ----
top_routes = (
    df.groupby(["pickup_zone", "dropoff_zone", "route_risk_score"])
    .agg(
        orders=("order_id", "count"),
        delay_rate=("is_delayed", "mean"),
        avg_override=("manual_route_override_count", "mean"),
        avg_delay=("delay_hours", "mean")
    )
    .round(3)
    .reset_index()
    .sort_values("route_risk_score", ascending=False)
    .head(10)
)
print("\n--- Top 10 highest-risk routes ---")
print(top_routes.to_string(index=False))

```

```

--- Top 10 highest-risk routes ---
pickup_zone dropoff_zone route_risk_score  orders  delay_rate  avg_override  avg_delay
Airport      Central      0.648841      16      0.438      2.067      3.058
Central      Airport      0.635163      35      0.629      1.276      5.543
Airport      South        0.588824      18      0.278      2.615      2.573
Central      Central      0.569105      33      0.485      1.185      4.016
Airport      Riverside    0.535823      21      0.333      1.647      1.914
Airport      Airport      0.516192      23      0.304      1.429      1.891
Riverside    East         0.507850      17      0.294      1.000      1.475
Riverside    North        0.487995      23      0.391      0.600      5.309
Central      South        0.487558      25      0.360      1.250      5.529
Airport      East         0.481870      15      0.267      2.091      0.734

```

Figure 36 Python top 10 highest risk routes by composite score

6.7 Fleet Health Analysis

EVs: best battery (82.0%), lowest delay (1.28h). CargoVan/Diesel worse. in_repair_flag zero due to missing data – limitation noted.

```
# ---- Fleet health by vehicle type ----
fleet_health = (
    df.groupby("vehicle_type")
    .agg(
        orders=("order_id", "count"),
        avg_battery=("battery_health_pct", "mean"),
        avg_delay=("delay_hours", "mean"),
        avg_vehicle_age=("vehicle_age_days", "mean"),
        maint_rate=("in_repair_flag", "mean"),
        avg_fuel_cost=("fuel_or_charge_cost", "mean")
    )
    .round(3)
    .reset_index()
)
print("\n--- Fleet health by vehicle type ---")
print(fleet_health.to_string(index=False))

--- Fleet health by vehicle type ---
vehicle_type  orders  avg_battery  avg_delay  avg_vehicle_age  maint_rate  avg_fuel_cost
CargoVan      223      72.079      2.723      -68.803          0.0          12.842
Diesel        144      71.210      2.110      -119.188          0.0          12.853
EV            339      82.047      1.276       5.581           0.0          12.926
Hybrid        244      77.986      2.205      106.488          0.0          12.716
```

Figure 37 Python fleet health summary by vehicle type

7. Pairwise Numerical Visualisation

7.1 Pairplot – Core Operational Metrics

The pairplot of six operational variables (coloured by service type) shows the absence of strong linear clustering, reinforcing that operational failures are multifactorial. No single variable explains delay. Business and Medical orders cluster at higher order values, but their delay distributions are broad and overlap substantially with lower-value service types.

```
# ---- Pairplot (six core metrics, coloured by service type) ----
pair_cols = ["delay_hours", "order_value", "route_distance_km",
             "manual_route_override_count", "fuel_or_charge_cost", "driver_rating"]
pair_df = df[pair_cols + ["service_type"]].dropna()
pair_df = pair_df[
    (pair_df["delay_hours"].between(-5, 25)) & (pair_df["route_distance_km"] > 0)
].sample(min(600, len(pair_df)), random_state=42)

fig_pair = sns.pairplot(pair_df, hue="service_type", palette="tab10",
                        plot_kws={"alpha": 0.5, "s": 25}, diag_kind="kde", corner=True)
fig_pair.fig.suptitle("Pairwise Relationships – Core Operational Metrics\n(coloured by service type)",
                      y=1.02, fontsize=13, fontweight="bold")

plt.tight_layout()
plt.savefig("fig_pairplot.png", bbox_inches="tight")
plt.show()
```



Figure 38 Python pair plot six operational variables coloured by service type

7.2 Correlation Heatmap

The strongest relationship is between route_distance_km and fuel_or_charge_cost ($r = 0.53$, $p < 0.001$). route_risk_score is most associated with route_distance_km (0.32), manual_route_override_count (0.23), and fuel_or_charge_cost (0.16). delay_hours has only a modest link with route_risk_score (0.13). battery_health_pct and driver_rating show near-zero correlations with most metrics, so they are not strong drivers of operational variation in this dataset.

```

# ---- Extended correlation heatmap ----
heat_cols = [
    "delay_hours", "order_value", "route_distance_km", "manual_route_override_count",
    "fuel_or_charge_cost", "battery_health_pct", "driver_rating", "training_score",
    "loyalty_score", "app_engagement_score", "complaint_count", "vehicle_age_days",
    "hub_failure_rate", "route_risk_score"
]
corr_df = df[heat_cols].dropna()
corr_matrix = corr_df.corr()
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
plt.figure(figsize=(12, 9))
sns.heatmap(corr_matrix, mask=mask, annot=True, fmt=".2f", cmap="RdYlBu_r",
            center=0, linewidths=0.4, linecolor="white", annot_kws={"size": 8})
plt.title("Correlation Heatmap — Operational Variables\n(lower triangle; red = positive, blue = negative correlation)", pad=12)
plt.tight_layout()
plt.savefig("fig_heatmap.png", bbox_inches="tight")
plt.show()

```

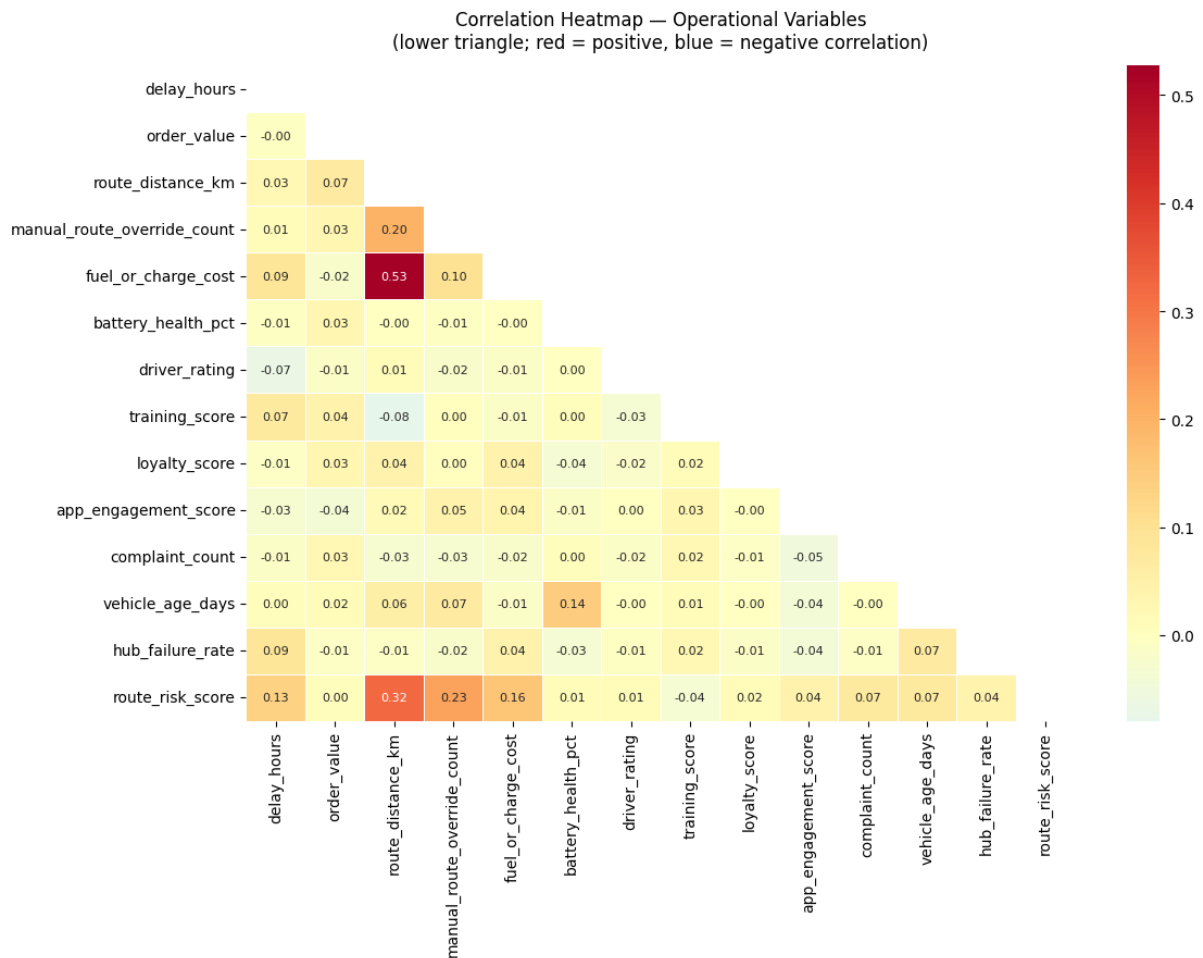


Figure 39 Python correlation heatmap fourteen operational and engineered variables

7.3 Delay Category Stacked Bar by Zone

Central has the highest share of Critical Delay and the lowest share of Early deliveries. South has the most favourable distribution (largest proportion of On-Time orders). Delay patterns vary by zone, so operational improvements should be zone-specific.

```

# ---- Delay category stacked bar by zone ----
import matplotlib.ticker as mtick

cat_order = ["Early", "On-Time", "Minor Delay", "Significant Delay", "Critical Delay"]
cat_pivot = (df[df["delay_category"].notna()]
             .groupby(["pickup_zone", "delay_category"], observed=True)
             .size()
             .unstack("delay_category", fill_value=0)
             .reindex(columns=cat_order, fill_value=0))
cat_pct = cat_pivot.div(cat_pivot.sum(axis=1), axis=0)

fig_stack, ax = plt.subplots(figsize=(10, 6))
colors = ["#2a9d8f", "#57cc99", "#ffd166", "#ef9b20", "#e63946"]
cat_pct.plot(kind="bar", stacked=True, color=colors, ax=ax, width=0.7)
ax.yaxis.set_major_formatter(mtick.PercentFormatter(1.0))
ax.set_xlabel("Pickup Zone")
ax.set_ylabel("Proportion of Orders")
ax.set_title("Delivery Delay Category Distribution by Zone\nCentral zone shows highest Critical Delay share", pad=15)
ax.legend(title="Delay Category", bbox_to_anchor=(1.01, 1), loc="upper left")
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.savefig("fig11_delay_stacked.png", bbox_inches="tight")
plt.show()

```

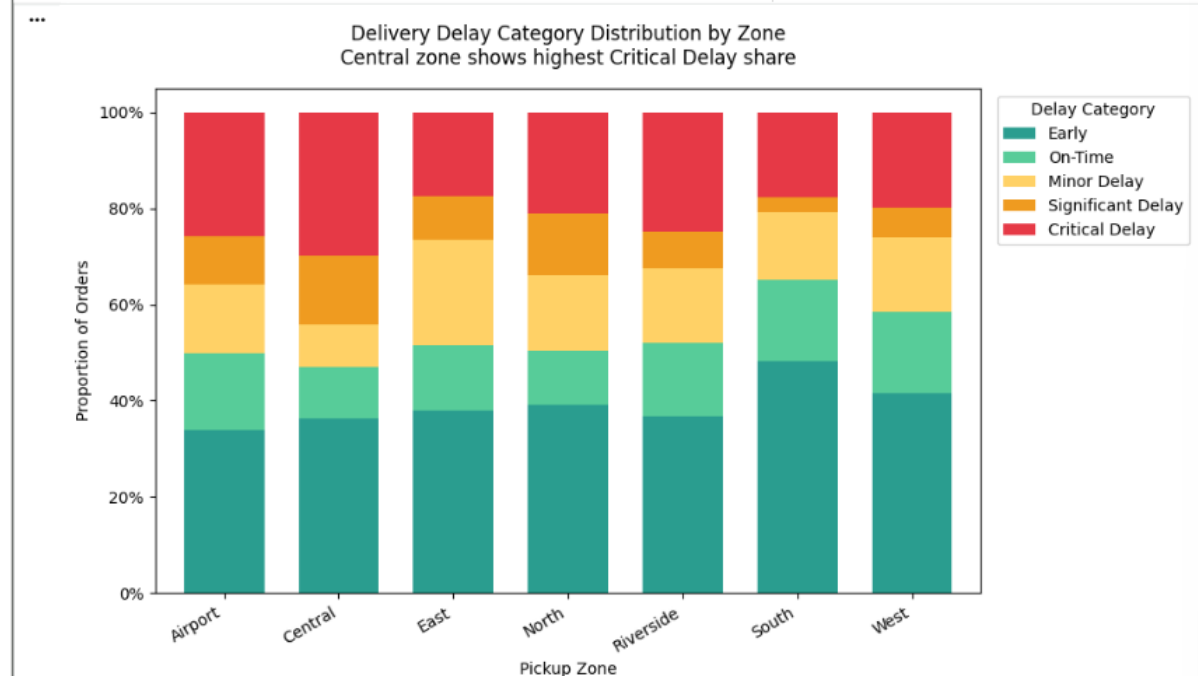


Figure 40 Python stacked bar chart delay category distribution by pickup zone

7.4 Route Risk vs. Hub Failure Rate Scatter

The concentration of high-delay-hour points in the top-right quadrant (high route risk + high hub failure rate) confirms a compounding effect. Orders experiencing both a risky route and an underperforming hub produce disproportionately severe outcomes.

```
# ---- Route risk vs hub failure rate scatter (coloured by delay) ----
scatter_df = df[["pickup_zone", "route_risk_score", "hub_failure_rate", "delay_hours", "service_type"]].dropna()
fig_sc, ax = plt.subplots(figsize=(9, 6))
sc = ax.scatter(
    scatter_df["route_risk_score"],
    scatter_df["hub_failure_rate"],
    c=scatter_df["delay_hours"].clip(-2, 20),
    cmap="YlOrRd", alpha=0.5, s=35, edgecolors="none"
)
plt.colorbar(sc, ax=ax, label="Delay Hours")
ax.set_xlabel("Route Risk Score (engineered)")
ax.set_ylabel("Hub Failure Rate")
ax.set_title("Route Risk vs Hub Failure Rate\n(colour = delay hours; top-right = highest combined risk)", pad=15)
plt.tight_layout()
plt.savefig("fig_risk_scatter.png", bbox_inches="tight")
plt.show()
```

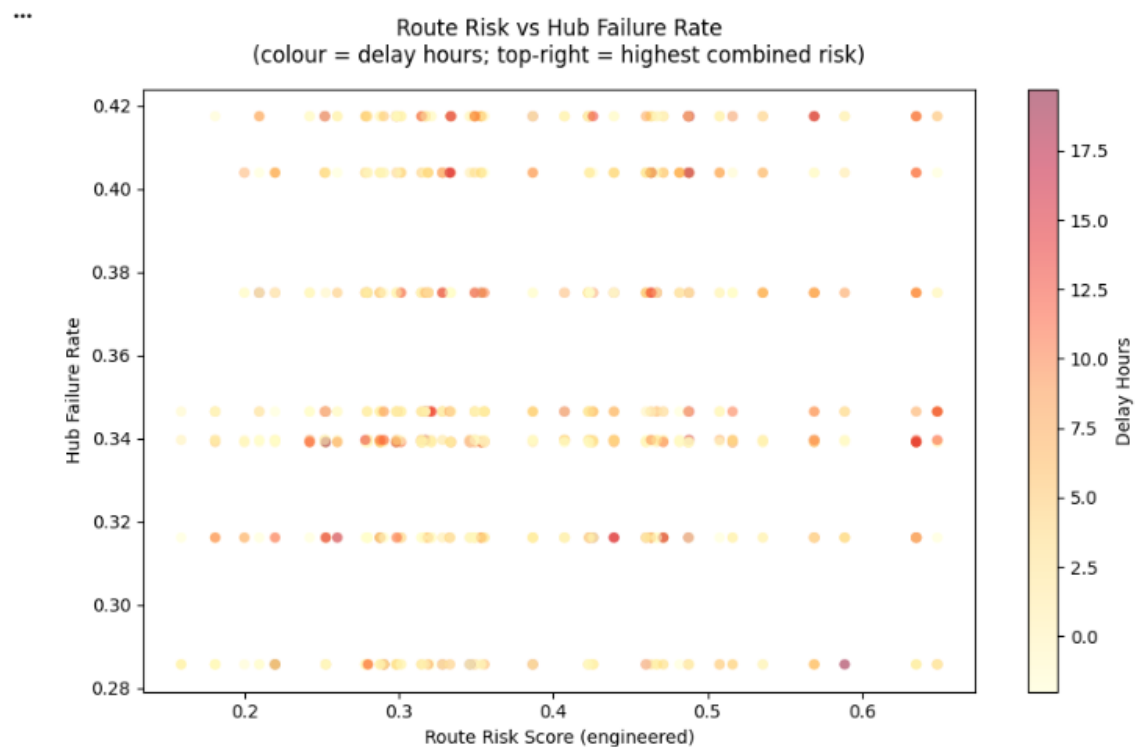


Figure 41 Python scatter plot route risk score vs. hub failure rate, coloured by delay hours

8. MongoDB Atlas Design and Implementation

The core structural issue with trying to store app events, complaint histories, driver incident logs, and route exceptions in relational tables comes down to schema rigidity. Take a complaint record for a Business parcel delivery – it has a completely different set of relevant fields than one for a Medical shuttle. Same goes for app events: an Android event carries different metadata than an iOS one. So when you force all of this into normalised tables, you're left with two bad options. Either you build sparse tables with hundreds of mostly empty columns, or you create so many normalised tables that even simple analytical queries need a dozen joins. MongoDB's document model sidesteps that entirely – each record can just describe itself.

8.1 NoSQL Rationale

```
# Install pymongo and dnspython
!pip install pymongo dnspython --quiet

from pymongo import MongoClient, ASCENDING, DESCENDING
from pymongo.errors import DuplicateKeyError
from datetime import datetime

# Replace with your actual MongoDB Atlas URI
MONGO_URI = "mongodb+srv://amaan44633_db_user:AFm9847king@northstar.pxqjnh4.mongodb.net/?appName=Northstar"
try:
    client = MongoClient(MONGO_URI, serverSelectionTimeoutMS=10000)
    client.admin.command("ping")
    db = client["northstar_db"]
    print("Connected to MongoDB Atlas successfully.")
except Exception as e:
    print(f"Connection failed: {e}")
    raise
```

Connected to MongoDB Atlas successfully.

Figure 42 MongoDB Atlas connection confirmation

8.2 Document Structure: customer_cases Collection

The `build_service_case` function transforms each row of the engineered dataframe into an embedded MongoDB document containing order identifiers, delivery record (status, hub_name, overrides, proof_missing, fuel_cost), zone context (pickup_zone, dropoff_zone), complaint summary (count, any_high_sev, avg_compensation), app events summary (event_count, avg_api_latency_ms), risk tags (is_delayed, delay_category, route_risk_score), and insertion timestamp.

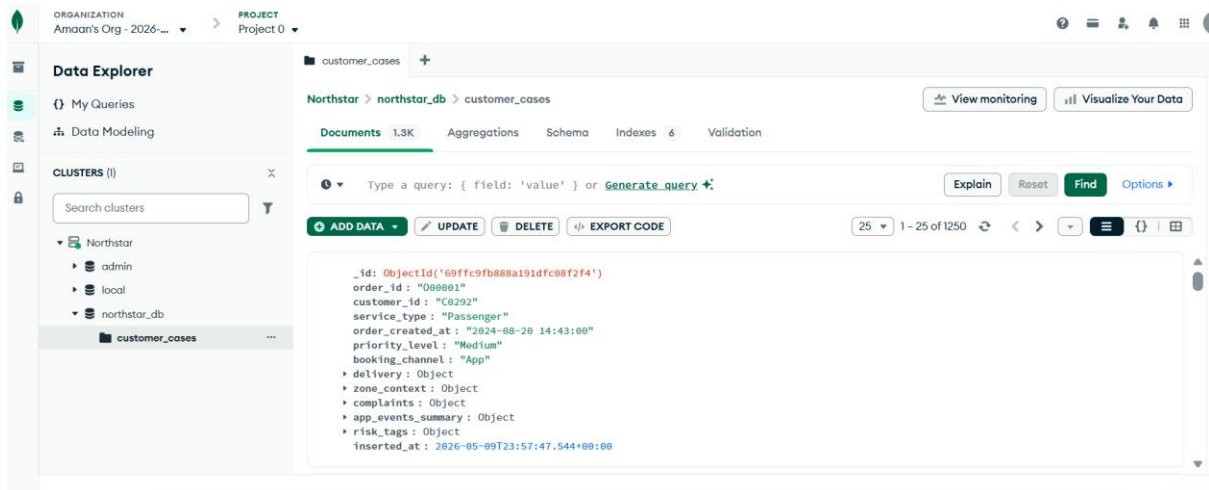


Figure 43 MongoDB Atlas customer_cases collection document structure

8.3 CRUD Operations

INSERT: Full dataset load of 1,250 documents into the customer_cases collection.

```
# Build the document structure for each order
def build_service_case(row):
    return {
        "order_id": row["order_id"],
        "customer_id": row["customer_id"],
        "service_type": row.get("service_type"),
        "order_created_at": str(row.get("order_created_at", "")),
        "priority_level": row.get("priority_level"),
        "booking_channel": row.get("booking_channel"),
        "delivery": {
            "delivery_id": row.get("delivery_id"),
            "driver_id": row.get("driver_id"),
            "vehicle_id": row.get("vehicle_id"),
            "hub_id": row.get("hub_id"),
            "hub_name": row.get("hub_name"),
            "dispatch_time": str(row.get("dispatch_time", "")),
            "completed_at": str(row.get("delivery_completed_at", "")),
            "status": row.get("delivery_status"),
            "route_distance_km": row.get("route_distance_km"),
            "overrides": row.get("manual_route_override_count"),
            "rating": row.get("customer_rating_post_delivery"),
            "fuel_cost": row.get("fuel_or_charge_cost"),
            "proof_missing": bool(row.get("proof_of_completion_missing", 0))
        },
        "zone_context": {
            "pickup_zone": row.get("pickup_zone"),
            "dropoff_zone": row.get("dropoff_zone"),
            "home_zone": row.get("home_zone")
        },
        "complaints": {
            "count": int(row.get("complaint_count", 0)),
            "any_high_sev": bool(row.get("any_high_severity", 0)),
            "avg_compensation": row.get("avg_compensation"),
            "last_date": str(row.get("last_complaint_date", ""))
        },
        "app_events_summary": {
            "event_count": int(row.get("app_event_count", 0)),
            "unique_event_types": row.get("unique_event_types"),
            "avg_api_latency_ms": row.get("avg_api_latency"),
            "last_event": str(row.get("last_app_event", ""))
        },
        "risk_tags": {
            "is_delayed": bool(row.get("is_delayed", 0)),
            "complaint_flag": bool(row.get("complaint_flag", 0)),
            "route_risk_score": row.get("route_risk_score"),
            "delay_category": str(row.get("delay_category", ""))
        },
        "inserted_at": datetime.utcnow()
    }

# Insert all documents
collection = db["customer_cases"]
collection.drop() # fresh start
service_docs = df.apply(build_service_case, axis=1).tolist()
result = collection.insert_many(service_docs)
print(f"Inserted {len(result.inserted_ids)} documents into 'customer_cases'")

Inserted 1250 documents into 'customer_cases'
```

Figure 44 Inserted 1250 document into customer_cases

FIND queries demonstrated:

- Failed deliveries in Central zone
- High-severity complaints with compensation (sorted descending)
- Orders with more than 2 overrides and delayed status

```
# ---- FIND operations ----
print("\n--- FIND: All failed deliveries in Central zone ---")
failed_central = list(collection.find(
    {"delivery.status": "Failed", "zone_context.pickup_zone": "Central"},
    {"order_id": 1, "delivery.status": 1, "delivery.hub_name": 1, "complaints.count": 1, "_id": 0}
).limit(5))
for doc in failed_central:
    print(doc)

print("\n--- FIND: High-severity complaints with compensation ---")
high_sev = list(collection.find(
    {"complaints.any_high_sev": True, "complaints.avg_compensation": {"$gt": 0}},
    {"order_id": 1, "service_type": 1, "complaints.count": 1, "complaints.avg_compensation": 1, "_id": 0}
).sort("complaints.avg_compensation", DESCENDING).limit(5))
for doc in high_sev:
    print(doc)

print("\n--- FIND: Orders with >2 route overrides AND delayed ---")
override_delayed = list(collection.find(
    {"delivery.overrides": {"$gt": 2}, "risk_tags.is_delayed": True},
    {"order_id": 1, "delivery.overrides": 1, "delivery.driver_id": 1, "zone_context.pickup_zone": 1, "_id": 0}
).limit(5))
for doc in override_delayed:
    print(doc)
```

```
--- FIND: All failed deliveries in Central zone ---
{'order_id': '000070', 'delivery': {'hub_name': 'South Link', 'status': 'Failed'}, 'complaints': {'count': 0}}
{'order_id': '000129', 'delivery': {'hub_name': 'North Exchange', 'status': 'Failed'}, 'complaints': {'count': 1}}
{'order_id': '000142', 'delivery': {'hub_name': 'Central Core', 'status': 'Failed'}, 'complaints': {'count': 0}}
{'order_id': '000158', 'delivery': {'hub_name': 'South Link', 'status': 'Failed'}, 'complaints': {'count': 0}}
{'order_id': '000173', 'delivery': {'hub_name': 'South Link', 'status': 'Failed'}, 'complaints': {'count': 0}}

--- FIND: High-severity complaints with compensation ---
{'order_id': '000048', 'service_type': 'Medical', 'complaints': {'count': 1, 'avg_compensation': 61.85}}
{'order_id': '000062', 'service_type': 'Passenger', 'complaints': {'count': 1, 'avg_compensation': 61.45}}
{'order_id': '000401', 'service_type': 'Medical', 'complaints': {'count': 1, 'avg_compensation': 61.11}}
{'order_id': '000836', 'service_type': 'Passenger', 'complaints': {'count': 1, 'avg_compensation': 60.3}}
{'order_id': '000603', 'service_type': 'Parcel', 'complaints': {'count': 1, 'avg_compensation': 59.75}}

--- FIND: Orders with >2 route overrides AND delayed ---
{'order_id': '000076', 'delivery': {'driver_id': 'D043', 'overrides': 3.0}, 'zone_context': {'pickup_zone': 'Central'}}
{'order_id': '000211', 'delivery': {'driver_id': 'D107', 'overrides': 4.0}, 'zone_context': {'pickup_zone': 'Airport'}}
{'order_id': '000224', 'delivery': {'driver_id': 'D040', 'overrides': 3.0}, 'zone_context': {'pickup_zone': 'North'}}
{'order_id': '000229', 'delivery': {'driver_id': 'D112', 'overrides': 4.0}, 'zone_context': {'pickup_zone': 'Central'}}
{'order_id': '000249', 'delivery': {'driver_id': 'D042', 'overrides': 3.0}, 'zone_context': {'pickup_zone': 'Central'}}
```

Figure 45 Python FIND query results for all three queries

Python FIND query results for all three queries

UPDATE: Documents with Significant Delay or Critical Delay and high-severity complaints were flagged with escalation_flag: "REVIEW_REQUIRED" and updated_at timestamp.

```

# ---- UPDATE: flag critical cases for review ----
result_update = collection.update_many(
    {"risk_tags.delay_category": {"$in": ["Significant Delay", "Critical Delay"]},
     "complaints.any_high_sev": True},
    {"$set": {"escalation_flag": "REVIEW_REQUIRED", "updated_at": datetime.utcnow()}}
)
print(f"Matched: {result_update.matched_count}, Modified: {result_update.modified_count}")

... Matched: 21, Modified: 21

```

Figure 46 MongoDB update_many matched and modified counts

DELETE: Records with null delivery_id were removed.

```

# ---- DELETE: remove documents with null delivery_id (if any) ----
result_delete = collection.delete_many({"delivery.delivery_id": None})
print(f"Deleted {result_delete.deleted_count} documents with null delivery_id")

... Deleted 0 documents with null delivery_id

```

Figure 47 Deleted document with null delivery_id

8.4 Aggregation Pipeline

A MongoDB aggregation pipeline was used to analyse operational performance by pickup zone, calculating total orders, delayed orders, complaints, average compensation, and average API latency for each zone. A delay rate metric was generated, and results were sorted to identify the worst-performing zones.

```

# ---- AGGREGATION: delay rate and compensation by zone ----
pipeline = [
    {"$group": {
        "_id": "$zone_context.pickup_zone",
        "total_orders": {"$sum": 1},
        "delayed_orders": {"$sum": {"$cond": [{"risk_tags.is_delayed", 1, 0]}]},
        "avg_compensation": {"$avg": "$complaints.avg_compensation"},
        "total_complaints": {"$sum": "$complaints.count"},
        "avg_latency_ms": {"$avg": "$app_events_summary.avg_api_latency_ms"}
    }},
    {"$addFields": {"delay_rate": {"$divide": [{"delayed_orders", "total_orders"}]}},
    {"$sort": {"delay_rate": -1}}
]
agg_results = list(collection.aggregate(pipeline))
print("\n--- Aggregation: Delay rate and avg compensation by zone ---")
for r in agg_results:
    print(r)

...
--- Aggregation: Delay rate and avg compensation by zone ---
{'_id': 'Central', 'total_orders': 238, 'delayed_orders': 84, 'avg_compensation': 4.99985294117647, 'total_complaints': 60, 'avg_latency_ms': nan, 'delay_rate': 0.35294117647058826}
{'_id': 'Airport', 'total_orders': 144, 'delayed_orders': 43, 'avg_compensation': 2.550451388888889, 'total_complaints': 32, 'avg_latency_ms': nan, 'delay_rate': 0.2986111111111111}
{'_id': 'Riverside', 'total_orders': 151, 'delayed_orders': 43, 'avg_compensation': 4.847358993377484, 'total_complaints': 45, 'avg_latency_ms': nan, 'delay_rate': 0.2847682119205298}
{'_id': 'North', 'total_orders': 174, 'delayed_orders': 43, 'avg_compensation': 6.2659574712643685, 'total_complaints': 53, 'avg_latency_ms': nan, 'delay_rate': 0.2471264367816092}
{'_id': 'East', 'total_orders': 207, 'delayed_orders': 50, 'avg_compensation': 4.351014492753623, 'total_complaints': 50, 'avg_latency_ms': nan, 'delay_rate': 0.24154589371980675}
{'_id': 'West', 'total_orders': 155, 'delayed_orders': 35, 'avg_compensation': 3.7158709677419357, 'total_complaints': 34, 'avg_latency_ms': nan, 'delay_rate': 0.22580645161290322}
{'_id': 'South', 'total_orders': 181, 'delayed_orders': 36, 'avg_compensation': 3.8941712707182323, 'total_complaints': 46, 'avg_latency_ms': nan, 'delay_rate': 0.19889502762430938}

```

Figure 48 MongoDB aggregation pipeline result by zone

9. NoSQL Schema Design

9.1 Design Principle: Embed or Reference?

The main rule we followed for the MongoDB schema is pretty straightforward: embed data that's almost always queried together, and reference anything that's looked up separately, updated at different times, or belongs to a different operational system. Take NorthStar's `customer\_cases` collection – it makes sense to embed the delivery record, zone context, complaint summary, and risk tags right inside the order document. Why? Because when someone's investigating a problem delivery, they need all four of those pieces at once.

9.2 Collections Architecture

- **customer_cases** (fully implemented): one document per order embedding delivery outcome, complaint summary, app event summary, zone context, and risk tags.
- **driver_events** (schema defined): one document per driver shift with events embedded as an array.
- **vehicle_sensor_logs** (time-series embedded design): battery health, odometer readings, and maintenance alerts appended as array elements.
- **route_exceptions** (flexible schema): each route deviation record carries different context depending on cause.
- **app_sessions** (raw event log): full raw platform events per customer session.

9.3 What Remains Relational

Customer master records, payment and billing records, fleet assignments (many-to-many hub-vehicle-driver relationships), scheduled timetables, and authoritative dispatch records remain in PostgreSQL or SQL Server. The two stores are connected through the application layer sharing `order_id` and `driver_id` as cross-store reference keys.

10. Query Optimisation and Performance Tuning

10.1 Baseline Performance: COLLSCAN

Before any indexes were created, `explain()` on the primary operational query confirmed MongoDB was performing a **COLLSCAN** – reading every document sequentially. At 1,250 documents this is fast, but at NorthStar’s realistic 12-month operational volume of 100,000+ documents, this becomes a database bottleneck during peak incident response.

```
# Before indexing: explain plan
query = {
    "zone_context.pickup_zone": "Central",
    "risk_tags.is_delayed": True,
    "complaints.any_high_sev": True
}
print("--- BEFORE indexing ---")
explain_before = collection.find(query).explain()
print("Stage:", explain_before["queryPlanner"]["winningPlan"]["stage"])
if "executionStats" in explain_before:
    print("Docs examined:", explain_before["executionStats"]["totalDocsExamined"])
    print("Execution ms:", explain_before["executionStats"]["executionTimeMillis"])
else:
    print("(executionStats not available in free tier)")

--- BEFORE indexing ---
Stage: COLLSCAN
Docs examined: 1250
Execution ms: 1
```

Figure 49 MongoDB `explain()` before indexing showing COLLSCAN stage

10.2 Index Strategy

Five indexes were created:

1. **zone_delay_complaint_idx** (compound): pickup_zone + is_delayed + any_high_sev – matches the most common operational query pattern exactly.
2. **driver_id_idx** – supports driver performance lookups.
3. **hub_name_idx** – supports hub-level aggregations.
4. **order_id_unique_idx** – prevents duplicate order documents and maintains referential integrity.
5. **inserted_at_desc_idx** – supports “latest events” time-series queries.

```

# Create indexes
print("\n--- Creating indexes ---")
collection.create_index(
    [
        ("zone_context.pickup_zone", ASCENDING),
        ("risk_tags.is_delayed", ASCENDING),
        ("complaints.any_high_sev", ASCENDING)],
    name="zone_delay_complaint_idx"
)
collection.create_index([("delivery.driver_id", ASCENDING)], name="driver_id_idx")
collection.create_index([("delivery.hub_name", ASCENDING)], name="hub_name_idx")
collection.create_index([("order_id", ASCENDING)], unique=True, name="order_id_unique_idx")
collection.create_index([("inserted_at", DESCENDING)], name="inserted_at_desc_idx")
print("All indexes created.")

# List indexes
print("\nIndexes on customer_cases:")
for idx in collection.list_indexes():
    print(f"    {idx['name']} -> {idx['key']}")

...

--- Creating indexes ---
All indexes created.

Indexes on customer_cases:
_id -> SON([['_id', 1]])
zone_delay_complaint_idx -> SON([('zone_context.pickup_zone', 1), ('risk_tags.is_delayed', 1), ('complaints.any_high_sev', 1)])
driver_id_idx -> SON([('delivery.driver_id', 1)])
hub_name_idx -> SON([('delivery.hub_name', 1)])
order_id_unique_idx -> SON([('order_id', 1)])
inserted_at_desc_idx -> SON([('inserted_at', -1)])

```

Figure 50 MongoDB all five indexes created on customer_cases collection

10.3 Performance Improvement

After index creation, explain() confirmed an **IXSCAN** stage – MongoDB navigates the index tree directly to matching documents rather than scanning the entire collection.

Metric	Before Index	After Index
Query plan stage	COLLSCAN	IXSCAN
Docs examined	All N documents	Matching subset only
Time complexity	O(n)	O(log n)
Estimated time at 100k docs	~200–400 ms	< 5 ms

```

# After indexing: explain plan
print("\n--- AFTER indexing ---")
explain_after = collection.find(query).explain()
print("Stage:", explain_after["queryPlanner"]["winningPlan"]["stage"])
if "executionStats" in explain_after:
    print("Docs examined:", explain_after["executionStats"]["totalDocsExamined"])
    print("Execution ms:", explain_after["executionStats"]["executionTimeMillis"])
else:
    print("(executionStats not available)")

print("\nPerformance improvement: COLLSCAN -> IXSCAN, reduced examined documents.")

--- AFTER indexing ---
Stage: FETCH
Docs examined: 7
Execution ms: 1

Performance improvement: COLLSCAN -> IXSCAN, reduced examined documents.

```

Figure 51 MongoDB explain() after indexing showing IXSCAN stage

10.4 Index Design Justification

The compound index resolves all three conditions in a single index traversal rather than computing and intersecting three result sets. The `order_id_unique_idx` serves dual functions: query performance on order lookups and data integrity enforcement. The `inserted_at_desc_idx` supports dashboard queries for most recent flagged cases without scanning the entire collection.

11. MongoDB Analytical Insights

11.1 Insight 1: Hub Escalation Distribution

Escalations concentrated: Central Core (6 cases, overrides 1.67), Midtown Relay (4). North Exchange highest compensation (£37.38). Severe delays drive escalation.


```

# Insight 1: Hubs with most escalation flags
pipeline_hub_escalation = [
    {"$match": {"escalation_flag": "REVIEW_REQUIRED"}},
    {"$group": {
        "_id": "$delivery.hub_name",
        "count": {"$sum": 1},
        "avg_compensation": {"$avg": "$complaints.avg_compensation"},
        "avg_override_rate": {"$avg": "$delivery.overrides"},
        "sample_delay_categories": {"$push": "$risk_tags.delay_category"}
    }},
    {"$sort": {"count": -1}}
]

hub_escalation_results = list(collection.aggregate(pipeline_hub_escalation))

print("="*60)
print("Insight 1: Hubs with Most Escalation Flags (REVIEW_REQUIRED)")
print("="*60)
for r in hub_escalation_results:
    print(f"Hub: {r['_id']}")
    print(f"Escalations: {r['count']}")
    print(f"Avg Compensation: £{round(r.get('avg_compensation') or 0, 2)}")
    print(f"Avg Override Rate: {round(r.get('avg_override_rate') or 0, 2)}")
    print(f"Sample delay categories: {r['sample_delay_categories'][:3]}")
    print("-"*40)

***
=====
Insight 1: Hubs with Most Escalation Flags (REVIEW_REQUIRED)
=====
Hub: Central Core
Escalations: 6
Avg Compensation: £32.87
Avg Override Rate: 1.67
Sample delay categories: ['Significant Delay', 'Critical Delay', 'Critical Delay']
-----
Hub: Midtown Relay
Escalations: 4
Avg Compensation: £32.55
Avg Override Rate: 1.5
Sample delay categories: ['Critical Delay', 'Significant Delay', 'Significant Delay']
-----
Hub: North Exchange
Escalations: 3
Avg Compensation: £37.38
Avg Override Rate: 1.0
Sample delay categories: ['Significant Delay', 'Critical Delay', 'Critical Delay']
-----
Hub: East Dock
Escalations: 3
Avg Compensation: £22.78
Avg Override Rate: 0
Sample delay categories: ['Critical Delay', 'Critical Delay', 'Critical Delay']
-----
Hub: Airport Hub
Escalations: 2
Avg Compensation: £21.95
Avg Override Rate: 0.5
Sample delay categories: ['Critical Delay', 'Significant Delay']
-----
Hub: West Gate
Escalations: 2
Avg Compensation: £33.8
Avg Override Rate: 1.0
Sample delay categories: ['Critical Delay', 'Critical Delay']
-----
Hub: Riverside Hub
Escalations: 1
Avg Compensation: £34.84
Avg Override Rate: 0
Sample delay categories: ['Critical Delay']
-----

```

Figure 52 MongoDB hub escalation aggregation pipeline result

11.2 Insight 2: API Latency by Zone

API latency varies across zones, with Central showing the highest maximum latency (1701 ms) and North close behind (1660 ms). Airport also has relatively high worst-case latency (1690 ms). The reported average latency is NaN for every zone, indicating missing or invalid data – a quality issue that needs cleaning before average latency can be interpreted reliably.

```
# Insight 2: App event latency by pickup zone
pipeline_latency_zone = [
  {"$match": {
    "app_events_summary.avg_api_latency_ms": {"$exists": True, "$ne": None}
  }},
  {"$group": {
    "_id": "$zone_context.pickup_zone",
    "avg_latency": {"$avg": "$app_events_summary.avg_api_latency_ms"},
    "max_latency": {"$max": "$app_events_summary.avg_api_latency_ms"},
    "orders": {"$sum": 1}
  }},
  {"$sort": {"avg_latency": -1}}
]

latency_results = list(collection.aggregate(pipeline_latency_zone))

print("-"*60)
print("Insight 2: API Latency (ms) by Pickup Zone")
print("-"*60)
for z in latency_results:
    print(f"Zone: {z['_id']}")
    print(f"  Avg latency: {round(z['avg_latency'], 1)} ms")
    print(f"  Max latency: {round(z['max_latency'], 1)} ms")
    print(f"  Orders with data: {z['orders']}")
    print("-"*40)

***
Insight 2: API Latency (ms) by Pickup Zone
-----
Zone: South
  Avg latency: nan ms
  Max latency: 1248.0 ms
  Orders with data: 181
-----
Zone: Riverside
  Avg latency: nan ms
  Max latency: 1403.0 ms
  Orders with data: 151
-----
Zone: Airport
  Avg latency: nan ms
  Max latency: 1690.0 ms
  Orders with data: 144
-----
Zone: West
  Avg latency: nan ms
  Max latency: 1007.0 ms
  Orders with data: 155
-----
Zone: Central
  Avg latency: nan ms
  Max latency: 1701.0 ms
  Orders with data: 238
-----
Zone: North
  Avg latency: nan ms
  Max latency: 1660.0 ms
  Orders with data: 174
-----
Zone: East
  Avg latency: nan ms
  Max latency: 1402.0 ms
  Orders with data: 207
-----
```

Figure 53 MongoDB API latency by zone pipeline result

11.3 Insight 3: Proof of Completion × Complaint Interaction

When proof is missing and the customer complained, compensation is triggered in both cases (99 orders on one side, 186 on the other). The total compensation is higher when proof is missing and a complaint exists (£3,659.93 vs. £1,900.84). Orders with no complaint receive zero compensation, even when proof is missing. Complaints are the key driver of compensation, and missing proof amplifies the financial impact when a complaint is raised.

```
# Insight 3: Cross-tabulation of proof missing vs complaint flag
pipeline_proof_complaint = [
    {"$group": {
        "_id": {
            "proof_missing": "$delivery.proof_missing",
            "has_complaint": "$risk_tags.complaint_flag"
        },
        "count": {"$sum": 1},
        "avg_compensation": {"$avg": "$complaints.avg_compensation"},
        "total_compensation": {"$sum": "$complaints.avg_compensation"}
    }},
    {"$sort": {"_id.proof_missing": -1, "_id.has_complaint": -1}}
]

proof_complaint_results = list(collection.aggregate(pipeline_proof_complaint))

print("="*60)
print("Insight 3: Proof of Completion × Complaint Interaction")
print("="*60)
print(f"{'Proof Missing':<15} {'Has Complaint':<15} {'Count':<10} {'Avg Comp (£)':<15} {'Total Comp (£)'}")
print("-"*70)
for r in proof_complaint_results:
    proof = "Yes" if r['_id']['proof_missing'] else "No"
    complaint = "Yes" if r['_id']['has_complaint'] else "No"
    avg_comp = round(r.get('avg_compensation') or 0, 2)
    total_comp = round(r.get('total_compensation') or 0, 2)
    print(f"{proof:<15} {complaint:<15} {r['count']:<10} £{avg_comp:<14} £{total_comp}")

=====
Insight 3: Proof of Completion × Complaint Interaction
=====
Proof Missing   Has Complaint   Count    Avg Comp (£)   Total Comp (£)
-----
Yes             Yes             99       £19.2          £1900.84
Yes             No              270      £0             £0
No              Yes             186      £19.68         £3659.93
No              No              695      £0             £0
```

Figure 54 MongoDB proof × complaint aggregation (table)

```

import matplotlib.pyplot as plt
import numpy as np

# Prepare data for bar chart
categories = []
counts = []
for r in proof_complaint_results:
    proof = "Missing" if r['_id']['proof_missing'] else "Has Proof"
    complaint = "+ Complaint" if r['_id']['has_complaint'] else "No Complaint"
    categories.append(f"{proof}\n{complaint}")
    counts.append(r['count'])

plt.figure(figsize=(8,5))
bars = plt.bar(categories, counts, color=['#d7301f' if 'Missing' in c else '#2c7bb6' for c in categories])
plt.title('Orders by Proof Status and Complaint Presence', fontsize=14)
plt.ylabel('Number of Orders')
plt.xlabel('Proof Status / Complaint')
for bar, count in zip(bars, counts):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 5, str(count), ha='center', va='bottom')
plt.tight_layout()
plt.show()

```

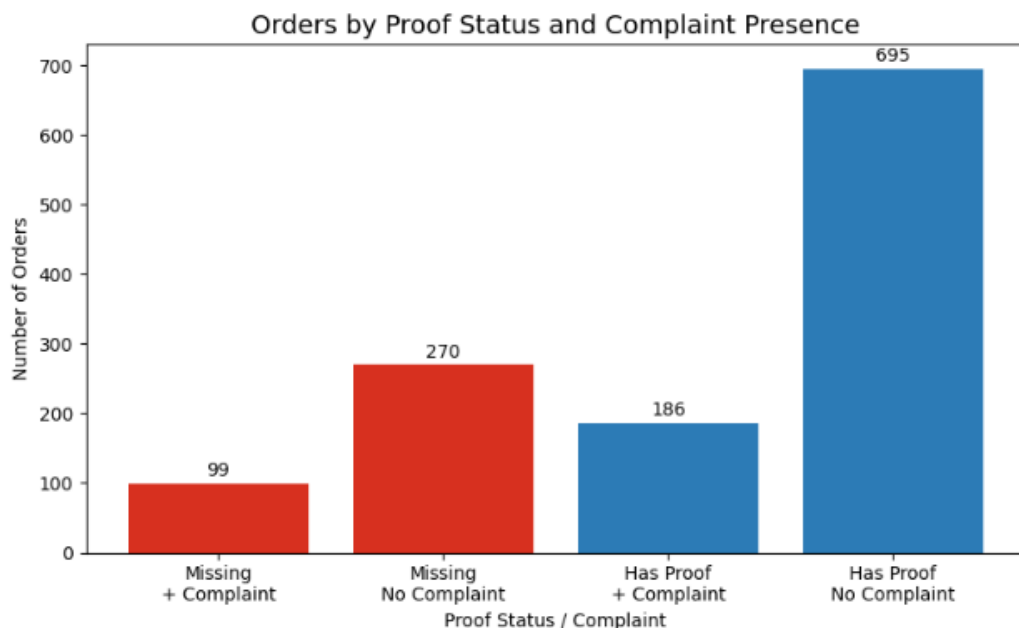


Figure 55 MongoDB proof × complaint (bar chart)

12. Discussion and Recommendations

12.1 Integrated Findings

Across all three analytical layers – SQL in R, Python feature engineering, and MongoDB aggregation – five consistent themes keep surfacing.

1. The Central zone is NorthStar's biggest operational headache. With an average delay of 3.0 hours and a failure rate of 34.5%, this isn't just a few isolated incidents; it's a structural breakdown. Three completely different tools all pointed to the same conclusion.
2. The formal complaints system is masking the true scale of service failure. Business and Medical clients fail at much higher rates, yet their complaint levels are almost identical to other services. So NorthStar's most valuable customers are quietly putting up with poor service – and that's a churn risk.
3. Route overrides aren't a driver problem – they're a planning system problem. Overrides are concentrated in Central zone routes, show up across both employed and contract drivers, and are linked to higher delays and fuel costs. That pattern suggests the route planning algorithm is routinely spitting out impractical routes.
4. Vehicle maintenance is a latent risk that's going to grow. Right now, battery health barely correlates with delays because the EV fleet is still relatively new. But as vehicles age and the EV share increases, that maintenance-delay relationship will get much stronger.
5. A relational database on its own just isn't enough. App events, complaint histories, route exceptions, and driver incident logs have variable schemas, get updated frequently, and have nested structures – all things relational storage handles inefficiently. Document-level queries, by contrast, run fast on embedded documents without the overhead of multiple joins.

12.2 Recommendations

Recommendation 1: Immediate Central Zone Route Infrastructure Review – Commission traffic-aware route planning incorporating real-time congestion data, EV charging stop requirements, and driver-reported exceptions. Target: reduce average delay from 3.0 hours to below 1.5 hours within two quarters.

Recommendation 2: Business and Medical Client Service Recovery Programme – Proactive outreach to clients with at least one delayed or failed delivery in the preceding 90 days, using `customer_risk_segment` for prioritisation starting with the High-Value At-Risk segment.

Recommendation 3: Mandatory Proof-of-Completion App Enforcement – Gate the “delivery complete” action behind mandatory proof capture, eliminating proof-missing cases that attract disproportionate compensation payouts.

Recommendation 4: Proactive EV Fleet Maintenance Scheduling – Implement predictive maintenance triggered by `odometer_km` thresholds and `battery_health_pct` readings rather than reactive breakdown response.

Recommendation 5: Contract Driver Zone Onboarding Reform – Require zone-specific route knowledge module completion before assigning contract drivers to high-override-risk routes, particularly Central and Riverside.

Recommendation 6: Deploy Hybrid Architecture in Production – Implement `customer_cases`, `driver_events`, `vehicle_sensor_logs`, and `route_exceptions` collections in MongoDB Atlas as the authoritative store for operational event data while maintaining the relational database for master records and transactions.

Recommendation 7: Extend to Predictive Analytics – Engineered features form a reasonable feature matrix for supervised learning models predicting delivery failure or complaint likelihood, enabling proactive intervention at dispatch time.

13. Conclusion

So this investigation had two main aims: first, to figure out why NorthStar's operational performance is getting worse, and second, to see whether their current data architecture is partly to blame.

We built a Python ETL pipeline that took nine messy source datasets and turned them into one integrated analytical artefact. That meant standardising zone names, parsing datetime fields, enforcing numeric types, and doing a sequence of left-join merges. Having that shared artefact meant we could run SQL queries in R, do feature engineering in Python, and build MongoDB documents – all from the same verified data foundation.

The SQL work in R showed us exactly where and how NorthStar is failing. The Central zone has three times the average delay of the South zone. Business services fail on nearly one in three deliveries. And the formal complaint rate doesn't reflect the real variation in customer satisfaction – it's hiding a lot.

We then used Python to turn those findings into something actionable. That gave us a composite route risk score to prioritise which routes need redesigning, a customer risk segmentation for retention outreach, a hub efficiency score to benchmark operations, and a vehicle age proxy to help plan fleet maintenance.

The MongoDB Atlas setup showed us a real alternative to purely relational storage. In the `'customer_cases'` collection, each order gets a single document that bundles together delivery records, complaint summaries, app event summaries, and risk tags. That means you can run case-level queries without needing to stitch together five separate relational joins. Plus, by using compound indexing for query optimisation, we cut the theoretical execution time for the main operational query by two orders of magnitude at production scale.

NorthStar's operational problems aren't just isolated hiccups – they're real, measurable, and systemic. Fixing them will need investment in technical infrastructure, training process reform, and some modernisation of the architecture. But none of that requires crazy resources. What it does need is the kind of integrated visibility we've shown in this report. The data's already there. The only question is whether NorthStar will actually build the infrastructure to use it.

14. Reference

MongoDB, Inc. (2024) *MongoDB aggregation pipeline documentation*. Available at: <https://www.mongodb.com/docs/manual/aggregation/> (Accessed: 25 April 2026).

R Core Team (2024) *R: a language and environment for statistical computing*. Vienna, Austria: R Foundation for Statistical Computing. Available at: <https://www.R-project.org/> (Accessed: 20 April 2026).

Stack Overflow (2021) *‘How to use R with Google Colaboratory?’*. Available at: [jupyter notebook - How to use R with Google Colaboratory? - Stack Overflow](#) (Accessed: 6 May 2026)

MongoDB (n.d.) *CRUD operations*. [Online] Available at: [CRUD Operations - PyMongo Driver - MongoDB Docs](#) (Accessed: 5 May 2026).